

DUMPSTER FIRE

Tomo 1

Sistemas de juego

Armas, enemigos, economía, progresión, meta y nivel

Documentación técnica interna · uso del equipo de desarrollo

Estado: alpha · 23-09-2026

Base de código: rama **main**, commit **53ef237** · Unity 6000.3.13f1 · URP 17.3

Contenido

Las referencias a los documentos 01–07 (GDD, Overheat, Spawning, Game Loop, Progresión, QA, Deltas) remiten a la documentación alpha del 22-09-2026, que no forma parte de esta carpeta. Las rutas `archivo:línea` son relativas a `Assets/Scripts/` salvo que se indique otra cosa. Cuando un valor serializado en escena o prefab difiere del default del script, manda el de la escena.

PARTE 1 · ARMAS

1	Armas: arquitectura	4
2	Armas: catálogo	9
3	Firepoints automáticos	16

PARTE 2 · ENEMIGOS Y NIVEL

4	Catálogo de enemigos	20
5	Nivel y entorno	24

PARTE 3 · ECONOMÍA Y PROGRESIÓN

6	Economía de materiales y balance CSV	29
7	Power-ups temporales y Scrap	34
8	Level-up y stats automáticas	37
9	Stats del jugador	40
10	Meta, challenges y logros	45

PARTE 1

Armas

1 Armas: arquitectura

El jugador lleva hasta tres armas. En cada momento, una está en manual (la dispara el jugador, con munición y habilidad en Q) y las otras disparan solas en automático. Al vaciar la munición del manual, esa arma vuelve a automático y la siguiente pasa a manual. El heat de la run alimenta la curva de poder de cada arma. El catálogo por arma está en `31-Armas-catalogo.md` y los firepoints en `32-Firepoints-automaticos.md`.

Scripts

Script	Rol
<code>Managers/WeaponManager</code>	Slots, modo manual/automático, input, munición y ciclo. Vive en <code>player.prefab</code>
<code>Managers/WeaponBehaviourFactory</code>	<code>WeaponType</code> → clase concreta (<code>WeaponBehaviourFactory.cs:13-21</code>)
<code>Base/WeaponBehaviourBase</code> (<code>BasicProjectileWeapon</code>)	Base de las cinco armas: ticks, gasto de munición, fire interval, spawn de proyectiles
<code>Types/*Weapon</code>	<code>AutomaticCannon</code> , <code>RocketLauncher</code> , <code>Flamethrower</code> , <code>Mortar</code> , <code>RotatingBlade</code>
<code>Base/WeaponData</code> + <code>WeaponModels</code>	<code>ScriptableObject</code> y datos de runtime (<code>WeaponInstance</code>)
<code>Managers/WeaponMath</code>	Munición máxima, cooldown de habilidad, knockback, multiplicador de cadencia
<code>Managers/WeaponDamageResolver</code>	<code>WeaponDamageContext</code> : cadena de daño y crítico
<code>Managers/PlayerWeaponMountController</code> , <code>AutomaticWeaponMount</code>	Montajes y firepoints (doc 32)
<code>Projectiles/ProjectilePool</code> (+ <code>ProjectilePoolMember</code> , <code>Projectile</code>)	Pool de proyectiles
<code>Economy/WeaponCraftingService</code>	Subida de nivel y Advanced Tinkering (rutas A/B)
<code>Economy/WeaponStatsParser</code> , <code>Weapon/Editor/WeaponBalanceSheetApplier</code>	Dos caminos para llevar el balance a los assets
<code>Managers/WeaponLevelUpHandler</code>	Armas por level-up. Está en el prefab, pero <code>LevelUpOrchestrator</code> no lo llama (<code>LevelUpOrchestrator.cs:63-73</code>)

WeaponManager

- `MaxWeaponSlots = 3` (`WeaponManager.cs:8`). `AddWeapon` crea el `WeaponInstance` y el `behaviour`, lo registra en el mount controller y, si es la primera arma, la pone en manual (:106-123).
- `_startingWeapons` está vacío en `player.prefab` (`player.prefab:795`). Si hay un `RunStartWeaponChoice` activo, el manager no equipa nada; el arma inicial sale de ese menú (:249-262).
- `HeatManager.GetInstance()` se resuelve una sola vez, en `Awake` (:51). Si el `HeatManager` aparece después, las armas quedan sin heat (`Heat == null` → sin bonus).

Orden de Update (:57-79)

1. Sale si `timeScale ≤ 0` o si `GameManager` no está en `IsPlaying`.
2. Evalúa la pose del esqueleto (`PlayerAnimationDriver.EvaluatePoseForWeapons`) y vuelve a resolver el aim con el muzzle ya ubicado. Ver doc 32.
3. `RefreshWeaponModes` en los mounts.
4. `TickAutomatic` de cada arma en estado `Automatic` (:306-315).

5. `UpdateManualWeapon`: input (click izquierdo = fuego, Q = habilidad, cableado a `Mouse / Keyboard`, :366-411), tick del cooldown de habilidad, `TickManual` y la habilidad. Las armas con `IHoldActiveAbilityBehaviour` (el cohete) reciben `begin/tick/release` (:336-348).
6. Si la munición del manual llegó a 0, llama a `EndManualMode` (:350-351).
7. `UpdateManualCycle` (:414-425).

Munición y ciclo manual

- `StartManualMode(i)` pone el arma `i` en `Manual`, el resto en `Automatic`, y **rellena la munición** del arma entrante con `WeaponMath.GetMaxManualAmmo` (:428-450).
- La munición máxima es `BaseManualAmmo × AmmoMultiplier × LevelData.ManualAmmoMultiplier`. Si la ruta define `ManualAmmoOverride ≥ 0`, ese valor la pisa (`WeaponMath.cs:49-67`).
- `TrySpendManualAmmo` casi siempre se llama con `requireFullAmount:false`: un disparo o una habilidad salen enteros aunque quede menos munición que el costo (`WeaponBehaviourBase.cs:198-215`).
- **Bug: el cooldown de cambio de manual no existe.** `EndManualMode` pasa directo a `StartManualMode(next)` (:453-465), y `StartManualMode` deja `_manualCooldownTimer = 0` (:435). Nadie asigna un valor positivo a ese timer, así que `UpdateManualCycle` nunca avanza. `_manualCycleCooldown` (1.25) y `_singleWeaponCycleCooldown` (2.5) están serializados y la UI los lee (`GetManualCooldownNormalized`, :208-214), pero no hacen nada.
- Con una sola arma, al vaciarse la munición se rellena en el mismo frame: el manual es infinito.
- Con varias armas, el cambio es instantáneo.
- **El cooldown de habilidad solo avanza en manual.** `TickAbilityCooldown` se llama solo para el arma manual (:333). Un arma que usó Q y pasó a automático conserva el cooldown congelado hasta volver a manual.
- El estado `WeaponState.Cooldown` (`WeaponModels.cs:65`) no se usa.
- `RefillManualAmmoAndResetActiveCooldown` (powerup Full Heal) recarga todas las armas y pone en 0 los cooldowns (:274-287).

Habilidad (Q)

- `CanUseAbility` exige que el arma esté en manual, que el cooldown sea 0 y que haya munición > 0 si el costo es > 0 (:229-238).
- El cooldown es `SkillCooldown × (1 - AbilityCooldownReduction)`, con la reducción acotada entre 0 y 0.95 (`WeaponMath.cs:81-110`). Es fijo por asset: no baja con el nivel.
- El costo es `ActiveAbilityAmmoCost`, salvo en `AutomaticCannon` ruta A, que cuesta 80 fijo (`WeaponMath.cs:91-102`).

WeaponData (ScriptableObject)

`Assets/ScriptableObjects/WeaponSO/<Arma>.asset`, menú `ScrapWaves/Weapon Data` (`WeaponData.cs:158`).

Grupo	Campos
Identidad	<code>WeaponId</code> , <code>DisplayName</code> , <code>WeaponType</code> , <code>Icon</code> , <code>PresentationProfile</code>
Targeting	<code>AutoTargetingMode</code> (<code>ClosestInRange</code> , <code>RandomInRange</code> , <code>IgnoreCameraClosest</code>), <code>AutomaticAimConstraint</code> (<code>CameraCone</code> , <code>Full360</code> , <code>BodyForward180</code>), <code>ManualMode</code>
Base	<code>BaseDamage</code> , <code>BaseAttackRate</code> , <code>BaseRange</code> , <code>BaseKnockback</code> , <code>BaseManualAmmo</code> , <code>ActiveAbilityAmmoCost</code> , <code>SkillCooldown</code>
Tuning por tipo	<code>_specificTuning</code> (<code>SerializeReference</code>): <code>AutomaticCannonTuning</code> , <code>RocketLauncherTuning</code> , etc. (<code>WeaponData.cs:6-156</code>). <code>OnValidate</code> fuerza el tipo que corresponde a <code>WeaponType</code>
Niveles	<code>LevelData</code> (1–10), <code>PathA / PathB</code> (<code>WeaponUpgradePathData</code> : nombre, multiplicadores, <code>ManualAmmoOverride</code> , <code>LevelData</code> 6–10)

Grupo	Campos
Balance crudo	BalanceStats, UpgradeSpecificStats. Están vacíos en los cinco assets
Meta	UnlockedFromStart, Requirement (IUnlockable)

WeaponLevelData tiene tres multiplicadores: daño, cadencia y munición (WeaponModels.cs:33-40). Rangos, radios, cooldowns y crítico no escalan por nivel en ningún lado.

Resolver de daño

WeaponDamageContext (WeaponDamageResolver.cs:100-372) se crea una vez por disparo (BasicProjectileWeapon.CreateDamageContext, WeaponBehaviourBase.cs:327-338):

1. $\text{BaseDamage} \times \text{LevelData.DamageMultiplier} \times \text{PathData.DamageMultiplier} \times \text{stat DamageMultiplier}$ (:170-180).
2. Si es daño de habilidad, $\times \text{AbilityDamageMultiplier}$ (:182-186).
3. **Tirada de crítico, una por contexto** (:191-197). Todos los objetivos de una explosión o de un tick comparten el resultado. - Probabilidad: $\text{Clamp01}(\text{stat CriticalChance})$ (:412-416). Sale solo del stat del jugador: la fila "Critical chance %" del CSV de armas no se lee. - Multiplicador: $\text{max}(1, \text{CriticalDamage} \times \text{CritMultiplierOverride})$ (:195). Solo el cañón define override ($\times 2$).
4. Por objetivo: $\times \text{elite}(\text{stat EliteDamageMultiplier}, \text{si WeaponEnemyClassifier dice elite o boss}) \times \text{rango}(\text{Sharpshooter/CQB: } > 15 \text{ m o } < 10 \text{ m, :348-363}) \times \text{DamageScale} \times \text{escala adicional}$ (:306-346).
5. Redondea con un mínimo de 1 (:286). OnDamageResolved alimenta el StatAttributionPanel y los diagnósticos.

Knockback: $\text{BaseKnockback} \times \text{stat Knockback} \times \text{escala} \times \text{falloff} \times (1 + \sqrt{\text{daño}} \cdot 0.25)$ (WeaponMath.cs:113-122). Con BaseKnockback 0 (el lanzallamas), el knockback es 0 sea cual sea la escala.

El clamp de crítico en 0–1 es lo que hace que un stat CriticalChance inflado (ver 05-Progresion-crafting-y-meta.md) quede en crítico al 100 %, en vez de romper la tirada.

Sinergia con el heat

- HeatManager.NormalizedHeat va de 0 a 1 y es por tramos: 0–0.8 lineal en el primer tramo de puntos y 0.8–1 en el segundo (HeatManager.cs:220-236). Durante un Overheat la barra está al 100 %, o sea $h = 1$.
- Cada arma lee Heat.NormalizedHeat en su propio código. No hay una tabla central: los umbrales están en el tuning del asset o hardcodeados (detalle por arma en el doc 31).
- Hook genérico: $\text{GetFireInterval} = \text{GetFireIntervalWithoutHeat} / \text{GetHeatFireRateMultiplier}$ (WeaponBehaviourBase.cs:176-192). El default es $1 + 0.25 \cdot h$. Cohete y cañón lo pisan con 1; el mortero lo pisa con su curva por encima de 0.5. Lanzallamas y blade no pasan por GetFireInterval.
- Aparte del heat, durante el Overheat OverheatManager suma +0.5 a AttackSpeedMultiplier con SetRuntimeFireRateMultiplier(1.5) (OverheatManager.cs:36, :133, :158). Al terminar lo vuelve a 1 (:186). Ese método borra **todos** los modificadores TemporaryEffect antes de aplicar el nuevo (PlayerStats.cs:146-151).
- El VFX y el feedback reciben NormalizedHeat en WeaponPresentationContext / WeaponFeedbackContext. Es solo cosmético.

Niveles y Advanced Tinkering

- WeaponInstance.Level va de 1 a 10 (WeaponManager.cs:133). HasAdvancedPath es $\text{Level} \geq 6$ (WeaponModels.cs:63). El CSV define los niveles 1–5 (básico) y 6–10 por ruta.

- `WeaponMath.GetLevelData` busca primero en `PathX.LevelData` y, si no encuentra, cae al `LevelData` básico (`WeaponMath.cs:6-17`).
- **Subida de nivel:** `WeaponCraftingService.TryUpgradeWeapon`, de a un nivel desde `CraftingUI` (`CraftingUI.cs:262`). Bloquea el paso 5→6 si no hay ruta (`WeaponCraftingService.cs:79-80`).
- **Advanced Tinkering**, solo en nivel 5 sin ruta (:114-186): 1. Se ofrece una ruta. Si B está desbloqueada en meta (`SaveManager.IsPathUnlocked`), sale al 50 %; si no, siempre A (:127-128). Reabrir la estación no vuelve a tirar (:122-130). 2. Aceptar: gasta materiales, sube a 6 y llama a `ApplyUpgradePath` (:171-178). 3. Rechazar: gasta materiales, marca el rechazo y garantiza la otra ruta con +50 % de costo. Solo se puede rechazar una vez, y solo si la otra ruta está desbloqueada (:135-140, :181-185).
- `ApplyUpgradePath` ignora la llamada si `Level < 6` (`WeaponManager.cs:173-180`).
- Riesgo latente: `WeaponLevelUpHandler` → `TryAddOrUpgradeWeapon` → `UpgradeWeapon` sube de nivel sin gate de ruta (`WeaponLevelUpHandler.cs:84`). Si se vuelve a cablear, un arma puede pasar a nivel 6 con `SelectedPath None`. **Advanced Tinkering** exige nivel 5 exacto, así que esa arma quedaría sin ruta para siempre.

De dónde salen los números de nivel

Camino	Qué hace	Estado
<code>WeaponStatsParser.ImportAll</code> : en editor por menú y por <code>BalanceAutoImporter</code> ; en Play por <code>EconomyBootstrap</code> (<code>EconomyBootstrap.cs:60-61</code>)	Busca "Basic" en la columna 1 y "Total" en la columna 0 (<code>WeaponStatsParser.cs:80, :120</code>). El CSV tiene los dos en la otra columna, así que <code>ApplyBlock</code> limpia <code>BalanceStats</code> y sale (:77-82). Después, <code>SyncLegacyLevelData</code> reescribe el <code>LevelData</code> básico con todos los multiplicadores en 1 (:209-231)	Roto. Cada Play en editor deja el <code>LevelData</code> básico en 1
<code>WeaponBalanceSheetApplier</code> (menú <code>Tools/ScrapWaves/Apply</code> <code>Weapon Balance Sheet Snapshot</code>)	Snapshot hardcodeado del CSV: daño y munición de los niveles 1–5 y de las rutas 6–10 (<code>WeaponBalanceSheetApplier.cs:36-111</code>)	Las <code>PathX.LevelData</code> actuales coinciden con este snapshot. El <code>BaseDamage</code> de los assets no (cañón 5 contra 20)

Resultado: en los niveles 1–5 el arma no mejora, y las rutas escalan sobre un `BaseDamage` que no es el del CSV. Si el parser se arregla, :203 carga `ActiveAbilityAmmoCost` desde la fila "**Ability damage**", y :289 mapea "ability cooldown" a `StatType.BaseFireInterval`. Los dos son bugs latentes.

Pool de proyectiles

- `ProjectilePool` vive en `GameplayScene`. El prefab de jugador deja `_projectilePool` en null y la escena lo asigna por override. El manager lo pasa a cada behaviour por la factory (`WeaponManager.cs:296-302`).
- `ProjectilePool.prefab`: 64 prewarm en `Awake`, crecimiento permitido hasta 1024 y vida de 3 s (`ProjectilePool.cs:89-93`).
- Cuando se agota, `TryGet` devuelve null y el disparo no sale; no recicla el proyectil más viejo (:108-124).
- `TrySpawnProjectile` configura, aplica el contexto de daño y lanza. `TrySpawnExplosiveProjectile` suma explosión, velocidad y viaje máximo **después** de `Launch`, porque `Launch` resetea esos campos (:265-337, `Projectile.cs:255-298`).
- Vuelve al pool por vida, por viaje máximo, por golpe de sweep o por trigger (`Projectile.cs:418-540`), vía `DespawnOrDestroy` → `ProjectilePoolMember` → `Release`.
- VFX: `ExplosionRadiusVfxPool` (8 iniciales, 32 máx. en escena) y `WeaponVfxPool` por cue (`MaxSimultaneous` 8 por defecto).

Deuda / puntos abiertos

- Cooldown del ciclo manual muerto (`WeaponManager.cs:435, :453-465`). Con una sola arma, el manual es infinito.

- El cooldown de habilidad solo corre en manual (:333).
- El parser del CSV no importa nada y pisa el `LevelData` básico en cada Play. `BaseDamage` de los assets ≠ CSV. Rangos, cooldowns y crítico por nivel no tienen consumidor.
- `WeaponStatsParser.cs:203` (costo ← "Ability damage") y :289 (cooldown → `BaseFireInterval`) quedan listos para romper el día que el parser funcione.
- El crítico sale solo del stat del jugador y se tira una vez por disparo. La columna de crítico por arma del CSV se ignora.
- `SetRuntimeFireRateMultiplier` borra todos los `TemporaryEffect`, no solo el del Overheat.
- `HeatManager` se toma una sola vez en `Awake`.
- Si se recablea `WeaponLevelUpHandler`, puede dejar armas en nivel ≥ 6 sin ruta.
- `ExplosionRadiusVfxPool.TotalCount()` cuenta solo los inactivos (`ExplosionRadiusVfxPool.cs:87`), así que el tope de 32 no se aplica.
- `ProjectilePool` no tolera instancias destruidas: si desencola un null, `ActivateInstance` tira excepción (`ProjectilePool.cs:116`, :625).
- La vida única de 3 s puede cortar tiros lentos de rango largo antes del viaje máximo.

2 Armas: catálogo

Una sección por arma de producción. Cada una cubre el rol, el automático, el manual, la habilidad (Q), las rutas A/B y la sinergia con el heat. La arquitectura común (slots, munición, resolver, niveles) está en `30-Armas-arquitectura.md`.

Cómo leer los números

- **CSV** = `Assets/Data/Balance/balance_weapon_stats.csv`. Es la intención de balance por nivel.
- **Asset** = `Assets/ScriptableObjects/WeaponSO/<Arma>.asset`. Es lo que corre hoy.
- Las dos fuentes no coinciden en daño base, cadencia ni rangos. El importador del CSV no carga ningún valor (ver `30-Armas-arquitectura.md`, "Niveles"). Consecuencias en runtime:
- Niveles 1–5: `DamageMultiplier` 1 en todo `LevelData`, así que el daño es el `BaseDamage` del asset en los cinco niveles.
- Niveles 6–10: `PathA/PathB.LevelData` sí escalan. El multiplicador sale de dividir el daño del CSV por el daño base del CSV, pero se aplica sobre el `BaseDamage` del asset.
- La munición sí coincide, porque `BaseManualAmmo` del asset es igual a la base del CSV en las cinco armas.
- El cooldown de habilidad es el `SkillCooldown` fijo del asset. Ningún nivel lo baja.
- En las tablas, "A6→A10" es el rango del nivel 6 al 10 por la ruta A. La fila **Runtime** es lo que calcula el código hoy.
- `h` = `HeatManager.NormalizedHeat` (0–1, por tramos; 0.8 = fin del primer tramo).

Arma	Daño base CSV	<code>asset</code>			Munición	Costo Q	CD Q
AutomaticCannon	20	5	5 (no se lee)	25	200	20	6
RocketLauncher	70	25	0.9	20	40	10	8
Flamethrower	25	5	1 (no se lee)	7	100	40	14
Mortar	100	40	0.75	22	15	5	10
RotatingBlade	60	25	0.8 (no se lee)	4.4	50	8	5

AutomaticCannon

Rol. DPS de línea con crítico. El automático tira ráfagas cortas; el manual, ráfagas sostenidas.

Automático. Busca objetivo dentro de `BaseRange` 25 con `BodyForward180` (`AutomaticCannonWeapon.cs:145-146`, `:1403-1414`). Tira una ráfaga de 3 balas (`CannonAutoBurstCount`) separadas 0.1 s (`:1338-1351`). Antes de cada bala revisa el ángulo y corta la ráfaga si el objetivo salió del frente (`:1370-1375`). La cadencia sale de `CannonAutoBurstsPerSecond` $0.5 \times \text{AttackSpeed} \times \text{multiplicador de nivel}$ (`:422-437`). Eso es una ráfaga cada 2 s. No usa `GetFireInterval`, así que `BaseAttackRate` 5 no se lee.

Manual. Mientras se mantiene el click, dispara ráfagas de hasta 5 balas (`clamp(ceil(ammo),1,5)`), a 1 de munición por bala y 1 ráfaga/s (`:238-240`, `AutomaticCannonFireLogic.cs:6-15`). La dirección sigue la retícula durante la ráfaga (`:1423-1425`). La escala de daño es 1 plano: el manual no recibe el bonus de heat (`:263`). No existe un rango manual; la fila "Manual range" del CSV no se usa.

Habilidad. Se activa al presionar, no hay que mantener. Tira una escopeta de $20 + \text{floor}(h \cdot 100/5)$ balas, de 20 a 40 (`:321-324`), con dispersión de 11° (`CannonAbilityScatterRadius`, que pese al nombre es un ángulo). Gasta 20 de munición y acepta munición parcial (`:302`).

Crítico. El multiplicador es `CriticalDamage` $\times$ `CannonCriticalDamageMultiplierOverride` 2, y $\times$ 1.5 más en ruta B (`:339-345`).

Stat (CSV)	1	2	3	4	5	A6→A10	B6→B10
Daño por bala	20	25	30	35	40	45→100	300→600
Cadencia auto (ráfagas o balas/s)	0.75	0.85	0.95	1.05	1.25	3.5→10	0.5→1.9
Cadencia manual	1.0	1.1	1.2	1.3	1.5	6→11	0.75→2
Munición manual	200	220	240	260	280	400→600	40→80
CD habilidad (s)	6	5.5	5	4.5	4	3.75→2	3.75→2
Crítico %	5	10	15	20	25	30→60	35→90
Específico						40 balas/s (80 en 10)	Punto débil ×2.5→×6
Runtime daño	5	5	5	5	5	11.25→25	75→150

Ruta A — Continuous Fire.

- Automático: bala única, con intervalo dividido por $(1.25 + \text{floor}(h \cdot 100/2) \cdot 0.01) \times 3$. Da 1.875 balas/s en frío y 2.625 en caliente (:428-432, :387-395).
- Manual: $(1.25 + \text{bonus de heat}) \times 5$, de 6.25 a 8.75 balas/s. El 1.25 está hardcodedo aunque exista el campo del tuning (:377-385, :397-406).
- Habilidad: ráfaga de 40 balas/s (constante, :46) durante $2 + \text{floor}(h \cdot 2)$ s, entre 80 y 160 balas (:455-469). Cuesta 80 de munición, fijo en `WeaponMath.cs:96-99`.

Ruta B — Head Hunter.

- Automático: una línea perforante, hasta 10 objetivos, con -10 % de daño por cada objetivo extra (:512-520) y rango de 1000 (:527-530). El intervalo se divide por `HeadHunterAutoAttackSpeedMultiplier 0.2` (:434-435): **un tiro cada 10 s**.
- Manual: cadencia × 1.75 (:412-413). El daño es ×2 contra elites y ×3 contra bosses, hardcodedo (:502-510). El punto débil solo cuenta en tiros manuales (:254).
- Habilidad: carga de 1 s que frena el movimiento (:623-636) y después un disparo con perforación ilimitada. Ese disparo aplica siempre el multiplicador de punto débil, pegue o no en uno (:546-547).

Heat.

Uso	Fórmula	Línea
Daño automático (no en Head Hunter)	$1 + \text{clamp}(\text{floor}(h \cdot 100/25), 0, 3) \cdot 0.15 \rightarrow 1.00 / 1.15 / 1.30 / 1.45$ en 25/50/75 %	:348-359
Balas de la habilidad	+1 cada 5 %, sin tope (+20 al 100 %)	:362-369
Cadencia ruta A	+0.01 cada 2 % (hasta +0.50)	:382-393
Duración de la habilidad A	+1 s en $h \geq 0.5$ y otro en $h = 1$	:457
Punto débil	$\text{clamp}(5 + \text{floor}(h/0.2), 5, 10)$	:481-483

`GetHeatFireRateMultiplier` devuelve 1 (:336), pero da igual: el cañón nunca llama a `GetFireInterval`.

Problemas conocidos.

- La habilidad de la ruta A gasta 80 y puede dejar la munición en 0. Entonces `EndManualMode` saca el arma del manual (`WeaponManager.cs:350-351`), la habilidad deja de tickear y `_continuousFireActive` queda en true. La ráfaga pendiente sigue la próxima vez que el arma entre en manual.

- El automático de Head Hunter (un tiro cada 10 s) queda muy lejos del CSV, que pide 0.5→1.9 balas/s.
- `GetContinuousFireBonus` (:486-492, que incluye el bonus del nivel 10) no se llama nunca. Tampoco se usan `FireLineBurst` (:1243) ni `GetRemainingHealth` (:1767). El $\times 1.25$ de la ruta B en la habilidad base (:325) es inalcanzable.
- Valores hardcodedos que el CSV quiere escalar: el punto débil (5→10 en código, 2.5→6 en el CSV), las 40 balas/s (el CSV pide 80 en nivel 10) y el rango de Head Hunter.

RocketLauncher

Rol. Área media con knockback. Cohetes curvos en automático; el lock-on múltiple es la habilidad.

Automático. Elige un objetivo en `BaseRange` 20 y guarda su punto de impacto al empezar la volea; el cohete va a ese punto fijo (`RocketLauncherWeapon.cs:115`, :126, :422). La volea tiene `RocketAutoBaseRocketCount` 1 + los cohetes de heat, con 0.11 s entre cohetes (:421, :443-447). Explosión de radio 2.5, falloff 1.0 y velocidad $\times 0.3$ (asset). El cohete sube primero hasta el tope del jugador + 0.6 y sigue una Bézier cuadrática (:494, `AutomaticRocketTrajectory.cs:22-45`). El heat no acelera el automático (`GetHeatFireRateMultiplier` → 1, :327). Con 0.9 de cadencia, sale una volea cada ~1.11 s.

Manual. Un cohete recto por disparo, a 1 de munición, hacia `Spawn + aim × BaseRange` (:149, :155). Explosión de radio 4, falloff 0.65, velocidad $\times 0.6$. Intervalo = `GetFireInterval / (1 + h)` (:330-335).

Habilidad (mantener Q). Implementa `IHoldActiveAbilityBehaviour`.

- Cono de 90° en `BaseRange` (:531-537). Arranca con 5 locks y suma uno cada 0.15 s mientras se mantiene (:233-238, :511-516).
- Máximo de locks: $10 + \text{floor}(h \cdot 10)$, de 10 a 20 (:497-501). Cuando todos los enemigos ya tienen lock, un boss acepta hasta 5 y un elite 2 (:548-556).
- Al soltar: un cohete por lock, con daño $\times 1.5$, radio 2 y velocidad $\times 1.15$ (:298-306).
- Cuesta 10 de munición, y solo se cobra si hubo al menos un lock (:277-289).

Stat (CSV)	1	2	3	4	5	A6→A10	B6→B10
Daño	70	80	90	100	110	130→250	120→175
Explosión auto	1.0	1.125	1.25	1.375	1.5	2→3.5	0.75→1.25
Explosión manual	2.0	2.25	2.5	2.75	3.0	3.5→5	1.5→2
Rango auto / manual	20 / 40					25 / 50	25 / 50
Munición	40	45	50	55	60	70→100	70→100
CD habilidad	8	8	8	8	7	7→4	7→4
Knockback	1.5					6	0.75
Específico						Debuff 5→10 s, $\times 1.25 \rightarrow \times 3$	Cono $\times 2$ ($\times 5$ en 10)
Runtime daño	25	25	25	25	25	46.4→89.3	42.9→62.5

Ruta A — Kinetic explosion. Radio $\times 2$ en todos los modos (:363-364; el CSV implica $\times 1.33$) y falloff $\times 0.65$ (:372-373). Knockback $\times 3$, o $\times 0.5$ en la habilidad (:380). Cada cohete aplica un amplificador de daño recibido de $\times 1.2$ durante 5 s, fijo (:707-708). El CSV pide que escale de 5 a 10 s y de $\times 1.25$ a $\times 3$.

Ruta B — Fragmentation cap.

- Radio $\times 0.5$ en auto y manual (:365-366) y knockback $\times 0.75$ (:382). En auto suma +1 cohete (:353).
- Cada explosión suelta fragmentos en un cono de 45°, con alcance = radio $\times 4$ y daño $\times 1$ (:386-407).
- La habilidad no hace lock-on: al soltar sale un cohete por la mira que libera 20 cohetes de racimo a $\times 0.5$ de daño (:251-271, :393-398, :716-751).

Heat.

Uso	Fórmula	Línea
Cadencia manual	$\text{intervalo} / (1 + h) \rightarrow \text{hasta} \times 2$	:330-335
Cohetes extra en auto	+1 en $h \geq 0.25, 0.50$ y 0.75	:343-347
Máximo de locks	+ $\text{floor}(h \cdot 10)$	:499

Problemas conocidos.

- El manual usa el mismo `BaseRange` 20 que el automático; el rango manual de 40–50 del CSV no existe.
- `GetFirstActiveTarget` (:627-636) está muerto. Las ramas de ruta B en :536 y :542-546 son inalcanzables.
- `TickManual` usa `Spawn.position` (:155) sin chequear null, a diferencia de :112 y :172.
- Todos los números de ruta ($\times 2$, $\times 0.65$, 20 cohetes de racimo, 45° , $\times 1.2 / 5$ s) están hardcoded y no escalan por nivel.

Flamethrower

Rol. Control cercano por ticks, con burn o freeze según la ruta.

Automático. No usa targeting: quema hacia el frente del cuerpo, proyectado al plano (`FlamethrowerWeapon.cs`:241-250). Cono de 45° con hasta 64 objetivos por tick (:276-282). Rango = `BaseRange` 7 $\times$ área (:716-719). Tick de 1 s, que baja a 0.5 s con $\text{heat} \geq 75\%$ (:253-260). En el básico no aplica burn; solo las rutas lo aplican, y solo si el golpe entró (:287-292).

Manual. Es una manguera: una polilínea de 12 segmentos con radio 0.75 (:306-312, :721-724), no un cono. Gasta 10 de munición por segundo en cada frame, también entre ticks (:128-130). Cada tick de 0.5 s pega daño $\times 1$ y aplica burn siempre (:152-164). Rango = $7 \times \text{área} \times (1 + 0.75 \cdot h)$, de 7 a 12.25 m (:263-267).

Habilidad. Barrido de 360° con radio $6 \times \text{área}$ ($\times 1.2$ en ruta A, $\times 0.9$ en ruta B), daño $\times 2$, y burn o freeze a todo lo que alcanza (:180-208, :747-755). Cuesta 40.

Stat (CSV)	1	2	3	4	5	A6→A10	B6→B10
Daño por tick	25	35	45	55	65	75→150	80→200
Rango auto	3	3	3.5	3.5	4	4→7	4→7
Rango manual	12	13	14	15	16	17→22	17→22
Munición	100	120	140	160	180	200→300	200→300
Radio habilidad	6	8	10	12	14	16→24	16→24
CD habilidad	14	13	12	11	10	9→5	9→5
Específico						Charco 1→3 m, 3→5 s	Freeze 2→6 s
Runtime daño	5	5	5	5	5	15→30	16→40

Ruta A — Jellified fuel.

- Burn $\times 1.35$ y duración $\times 2$, o sea 6 s (:353, :726-732).
- Cada aplicación de burn deja un charco con radio $0.75 \times \text{levelScale}$ que dura lo mismo que el burn (:507-518). $\text{levelScale} = \max(1, \text{Level}/6)$ vale 1.0 en nivel 6 y 1.67 en nivel 10 (:509, :744).
- La habilidad deja un charco grande (:211-222).

Ruta B — Liquid nitrogen. Reemplaza el burn por un slow que baja de $\times 0.5$ a $\times 0.1$ en 6 golpes y dura 3 s (:542). La habilidad congela 2 s, fijo, y después aplica $\times 0.1$ durante 4 s (:531-535).

Heat. Tick automático de 0.5 s con $h \geq 0.75$ (`FlameOverheatTickThresholdPercent` 75, :255-256) y rango manual $\times (1 + 0.75 \cdot h)$ (:265-266). No recibe el bonus de cadencia base: usa sus propios timers.

Problemas conocidos.

- `BaseKnockback` 0 en el asset anula todo el knockback: las escalas 0.25 del manual y 3 de la habilidad multiplican por 0 (`WeaponMath.cs`:118-121).
- En manual, la ruta A crea un charco por objetivo por tick, hasta 64 cada 0.5 s y cada uno de 6 s (:507-518). Es un riesgo de rendimiento, y además cada charco suma su propio burn.
- El burn manual y el de la habilidad se aplican aunque el golpe directo no haya entrado (:208, :320-323).
- Con `FlameBurnDuration` ≤ 0 , el slow de la ruta B también se apaga, porque ese chequeo corre antes de la rama B (:472).
- `FlameManualConeAngle` y `ApplyBurnToTarget` (:455-463) no se usan.

Mortar

Rol. Artillería de área con arco, lenta y sin crítico (`MortarWeapon.cs`:223).

Automático. `RandomInRange`: toma un enemigo al azar dentro de `BaseRange` 22 (:72-76). Al punto de impacto le suma un desvío aleatorio de 3.8 m (:83, :498-505). Vuelo de 2 s con arco de altura 8, parábola $4 \cdot h \cdot t(1-t)$ (`MortarTrajectory.cs`:12). Explosión de radio $2 \times \text{área}$, con falloff 0.5 hacia el borde. Intervalo base ~ 1.33 s dividido por el multiplicador de heat.

Manual. El impacto cae siempre a `BaseRange` en la dirección de la mira, no donde apunta la retícula (:126). Desvío de 0.75 m, 1 de munición por proyectil. Vuelo de $1 / (1 + 0.5 \cdot h)$ s: 1 s en frío, 0.667 s con heat al máximo (:239-243). Explosión de radio 2.5. El heat no acelera el manual (:234-237).

Habilidad (bombardeo). Tira $5 + \text{floor}(h \cdot 10)$ proyectiles, de 5 a 15 (:245-249). Caen en vertical desde 14 m sobre un disperso de $6 \times \text{área}$ (:174, :197). Daño $\times 1.2$, radio 3.2, caídas escalonadas cada 0.08 s (:297). Cuesta 5 y acepta munición parcial (:170).

Stat (CSV)	1	2	3	4	5	A6→A10	B6→B10
Daño	100	125	150	175	200	100→200	225→400
Rango auto / manual	30 / 40					30 / 40	25 / 35
Cadencia auto / manual	0.25 / 0.5					0.33 / 0.5	0.33 / 0.5
Munición	15	16	17	18	20	21→30	21→30
Radio habilidad	4	4.25	4.5	4.75	5	7→9	7→8
CD habilidad	10					10→6	10→6
Específico						Explosiones 3→8	Proyectiles extra 15→30, 10/s
Runtime daño	40	40	40	40	40	40→80	90→160

Ruta A — Grapeshot (código). El proyectil ignora colisiones y revienta en el aire, a mitad de camino entre el apex y el impacto (`MortarShellImpact.cs`:432, :787-795). La explosión principal no hace daño; lo hacen 15 fragmentos a $\times 0.5$ (`MortarWeapon.cs`:261, `MortarShellImpact.cs`:554-560). En la habilidad cae una lluvia de $(10 + \text{floor}(h \cdot 10)) \times 5$ proyectiles comunes durante 5 s, de 50 a 100 en total (:276-295).

Ruta B — Multi-charged shells (código). El proyectil atraviesa lo dañable y explota 3 veces en el mismo punto, cada 2 s (:265, `MortarShellImpact.cs`:565-595). En la habilidad, todos los proyectiles caen juntos a los 0.3 s.

Heat.

Uso	Fórmula	Línea
Cadencia auto	1 hasta $h = 0.5$; después $1 + \text{InverseLerp}(0.5, 1, h) \cdot 0.75$, con tope $\times 1.75$	:225-232
Velocidad del manual	$1 + 0.5 \cdot h$	:241
Proyectiles de la habilidad	$+ \text{floor}(h \cdot 10)$	:247
Lluvia de ruta A	$+ \text{floor}(h \cdot 10)$ por segundo	:279

Problemas conocidos.

- Las rutas están cruzadas respecto del CSV. El CSV pone "Projectile explosions 3→8" bajo Grapeshot, y "Extra projectile number" y "10/s" bajo Multi-charged. El código hace lo contrario, con valores fijos (3 explosiones, 15 fragmentos, 10/s) que no escalan por nivel.
- Los fragmentos de Grapeshot tienen un radio de explosión de ~ 0.17 m (`MortarShellImpact.cs:666-700`) y casi no pegan. El `GrapeshotConeAngle` de 70° solo aparece en los diagnósticos.
- `MortarActiveShellCount` no se lee: el 5 está hardcodeado (:248). `WeaponBalanceSheetApplier.cs:133` también lo fuerza a 5.
- Un solo rango y una sola cadencia para los dos modos; el CSV distingue auto de manual.

RotatingBlade

Rol. Melee orbital. En automático la hoja gira alrededor del jugador; en manual da tajos en cono.

Automático. La hoja orbita a un radio de $2.2 \times \text{área}$, con radio de golpe 0.6 (`RotatingBladeWeapon.cs:395-406`). Los golpes se resuelven sobre el arco recorrido, en tramos de 20° (:268-283). Gira a $240^\circ/\text{s} \times \text{AttackSpeed} \times \text{multiplicador de nivel}$ (:223-229), unas 0.67 vueltas/s. No lee `BaseAttackRate`. El daño se aplica cada 0.25 / cadencia s (:231-237). Knockback $1.5 + 0.5 \cdot h$ (:389-393).

Manual. Tajo en cono de 85° con rango $4.4 \times \text{área}$ (:408-411). Cooldown de 0.85 / cadencia y 1 de munición por tajo (:239-245, :142). Daño $\times (1 + h)$, hasta $\times 2$ (:343-347).

Habilidad (estocada). Línea de ancho $0.8 \times \text{área}$, con daño $\times 1.5$ y knockback 1.25 . Largo = rango manual $\times \min(3.5 + \text{floor}(h \cdot 100/20), 10)$ (:413-428). Cuesta 8.

Stat (CSV)	1	2	3	4	5	A6→A10	B6→B10
Daño	60	70	80	90	115	55→65	215→295
Largo de la hoja auto	2	2.25	2.5	2.75	3	3.25→6	3.125→5
Vueltas/s auto	0.25	0.3	0.35	0.4	0.5	0.6→2	0.65→1.5
Rango manual	3	3.5	4	4.5	5	5.25→10	5.5→10
Munición	50	55	60	65	75	80→120	80→120
Largo habilidad	10	10.5	11	11.5	12	12.5→20	2→4
CD habilidad	5	5	4.5	4.5	4	4→2	2→1
Específico						Hojas extra 1→8	—
Runtime daño	25	25	25	25	25	22.9→27.1	89.6→122.9 ($\times 2$, ver problemas)

Ruta A — Multi-blade.

- Número de hojas: $1 + \max(1, \text{Level} - 5)$, o sea 2 a 6 entre los niveles 6 y 10 (:297). El CSV pide 1→8 hojas extra (9 en total en el nivel 10).
- Las hojas salen cada 0.1 s con 8° de abanico (:382, :772), todo hardcodeado.

Ruta B — Atomic sharpness.

- Daño $\times 2$, knockback 0 y giro $\times 1.5$ (:349-353).
- La estocada se reemplaza por un dash: rango base = rango manual $\times 3$, o sea 13.2 m (el CSV dice 2 m). Crece $\times (1 + \text{golpes})$ en hasta 4 pasadas (:371-380, :742-766).
- El dash da invulnerabilidad por su duración + 0.25 s (:366-369).

Heat. Daño manual $1 + h$ (:345), knockback automático $+0.5 \cdot h$ (:391) y largo de la habilidad con $+1.0 \times$ cada 20 % (:416). No afecta el daño automático, los intervalos ni el dash.

Problemas conocidos.

- Probable doble conteo en la ruta B: el `DamageMultiplier` de `PathB.LevelData` (3.58→4.92) ya reproduce el CSV, y :349 vuelve a multiplicar $\times 2$.
- `BladeActiveMaxRangeMultiplier` 10 es inalcanzable: el máximo real es $3.5 + 5 = 8.5$.
- En Multi-blade, las estocadas extra pegan sobre la misma línea (solo cambia el VFX, :619, :659-663). Una munición paga los N tajos y el cooldown se estira $\sim(N-1) \cdot 0.1$ s (:129-132).
- El dash usa 1.5 de daño hardcodedo (:355) en vez de `BladeActiveDamageScale`, e ignora `AtomicDashRangePerHitMultiplier` al calcular la velocidad (:722).
- `GetRemainingHealth` (:909-925) está muerto.

Assets Sandbox_*

Hay dos juegos de cinco `Sandbox_<Arma>.asset`, y no son iguales entre sí.

Carpeta	Uso
<code>Assets/Scripts/Weapon/Testing/SO/</code>	Los genera <code>WeaponTestingSandboxSceneBuilder</code> (<code>WeaponAssetFolder</code> , :15, :69-73) y los referencia <code>Scenes/Testing/WeaponTestingSandbox.unity</code> . Sus valores vienen del builder (por ejemplo, el cañón con daño 10 y rango 12) y no siguen el balance de producción.
<code>Assets/ScriptableObjects/WeaponSO/Sandbox_*</code>	Sin referencias en escenas, prefabs ni código. Son copias huérfanas; el cañón tiene daño 10, rango 25 y <code>SkillCooldown</code> 1.

Ninguno de los dos juegos está en `EconomyBootstrap` ni en los pools del prefab `player`. El sandbox también puede pisar stats en runtime con `WeaponStatOverride` y `WeaponHeatOverride` (ver 06-QA-y-debug.md).

Deuda / puntos abiertos

- Ningún número por nivel del CSV llega al juego en los niveles 1–5. Las cinco armas pegan igual en el nivel 1 que en el 5, y el daño base del asset está entre 25 % y 42 % del CSV.
- Los cooldowns de habilidad, los rangos, los radios, la cadencia por nivel y los stats específicos de ruta del CSV no tienen lectura en código. `UpgradeSpecificStats` está vacío en todos los assets.
- Los valores de las rutas están hardcodedos en cada `*Weapon.cs` y no escalan del nivel 6 al 10, salvo `levelScale` del lanzallamas y el conteo de hojas del blade.
- Las rutas del mortero están cruzadas respecto del CSV.
- El automático de Head Hunter dispara cada 10 s.
- El lanzallamas tiene knockback 0.
- En Atomic sharpness, el daño se multiplica dos veces.
- Borrar o documentar el juego huérfano de `ScriptableObjects/WeaponSO/Sandbox_*`.

3 Firepoints automáticos

Cada arma equipada se ve puesta en el cuerpo del jugador (un "wearable"). Mientras está en automático dispara desde el muzzle de ese montaje. El arma manual dispara desde la mano derecha (Main Weapon Fire Point). El sistema tiene que garantizar que el muzzle esté en la pose final del frame antes de que un arma lo lea.

Scripts

Script / asset	Rol
Managers/PlayerWeaponMountController	Crea un montaje por <code>WeaponType</code> , decide el muzzle de cada arma según su modo y destruye los montajes al limpiar
Managers/AutomaticWeaponMount	Componente del prefab del montaje: Muzzle, visibilidad, luces indicadoras de auto y sink de aim (sin efecto en wearables)
Managers/WearableWeaponMountCatalog + Assets/Resources/WearableWeaponMounts.asset	<code>WeaponType</code> → prefab, <code>AttachmentPath</code> y offsets locales
Base/WeaponFireOrigin (WeaponFireOriginBinding, IWeaponFireOriginReceiver)	Par muzzle + sink de aim que recibe cada behaviour
Player/Animation/PlayerAnimationDriver	Evalúa la pose antes de los ticks de arma y hace que los wearables sigan a sus huesos

Cómo se asigna un montaje

1. `WeaponManager.Awake` inicializa el controller con el projectile spawn de la mano (`WeaponManager.cs:45-49`). Si el componente falta, lo agrega con `AddComponent`. En `player.prefab` viene configurado (`player.prefab:797-799`), así que ese fallback no corre en `GameplayScene`.
2. Si `_catalog` está vacío, `Initialize` carga `Resources/WearableWeaponMounts` (`PlayerWeaponMountController.cs:52-53`).
3. `WeaponManager.AddWeapon` → `_mountController.AddWeapon(behaviour, manual: primera arma)` (`WeaponManager.cs:117`).
4. `AddWeapon` busca `_mounts[WeaponType]` o llama a `CreateMount(type)` (`PlayerWeaponMountController.cs:56-77`). La clave es el tipo, no el índice de slot: dos armas del mismo tipo compartirían montaje.
5. `CreateMount` pide la entrada al catálogo (`WearableWeaponMountCatalog.cs:25-38`, búsqueda lineal que salta prefabs null) y resuelve el anclaje (`PlayerWeaponMountController.cs:177-223`).

Mapeo a huesos

`TryResolveAttachment` (`PlayerWeaponMountController.cs:205-223`):

- Si existe un socket animado en `_animatedSockets` para ese tipo, gana: el montaje queda en el socket con transform local identidad.
- Si no, usa el `AttachmentPath` y los offsets del catálogo. En `WearableWeaponMounts.asset` todos los `AttachmentPath` están vacíos, lo que significa la raíz del jugador. En la práctica mandan los sockets.

Sockets de `player.prefab` (`player.prefab:1228-1238`), creados por `Editor/PlaceholderPlayerAnimationBuilder.cs:170-171`:

Arma	Hueso del socket
AutomaticCannon	<code>shoulder.R</code>

Arma	Hueso del socket
RocketLauncher	spine.003 (pecho)
Mortar	spine.003 (pecho)
Flamethrower	forearm.L
RotatingBlade	spine
Arma manual (cualquiera)	Main Weapon Fire Point, hijo de hand.R

Cohete y mortero comparten hueso: si los dos están equipados, sus muzzles salen de la misma zona del pecho.

Manual vs automático

RefreshWeaponModes → ApplyMode (PlayerWeaponMountController.cs:89, :131-139):

Modo	Muzzle	Sink de aim
Automático	mount.Muzzle	el montaje
Manual	_mainFirePoint (mano)	ninguno

- SetFireOrigin solo rebindea si cambia el muzzle o el sink (:141-146, WeaponBehaviourBase.cs:70-77).
- Al rebindear, el cañón cancela la ráfaga en vuelo y el cohete cancela la volea (AutomaticCannonWeapon.cs:103-118, RocketLauncherWeapon.cs:90-95). O sea, al cambiar de manual se pierde lo que estaba saliendo.
- El modelo del montaje sigue visible aunque el arma esté en manual; solo se apagan las luces AutoIndicator* (AutomaticWeaponMount.cs:74-109).
- RefreshWeaponModes corre dos veces por frame: una desde WeaponManager.Update y otra en el LateUpdate del controller (PlayerWeaponMountController.cs:129).

Aim del montaje

WeaponFireOriginBinding.AimAt / AimAlong / ClearAim reenvía al sink (WeaponFireOrigin.cs:23-36).

BasicProjectileWeapon.TickAutomatic llama a AimAt si tiene objetivo y a ClearAim si no (WeaponBehaviourBase.cs:119-126).

En los wearables no rota nada. TickAim sale enseguida (AutomaticWeaponMount.cs:159-160), los cinco prefabs tienen _wearable: 1 y _aimPivot vacío, y el recoil del montaje también se saltea. La dirección del disparo la calcula cada arma, desde la posición del muzzle hacia el objetivo. El camino de torreta (no wearable: _turnSpeedDegrees, ApplyWeaponColor) queda sin uso.

Pose antes del disparo

Orden dentro de WeaponManager.Update (WeaponManager.cs:66-77):

1. ClearManualWeaponOverride(): limpia el override manual que usa el sandbox.
2. EvaluatePoseForWeapons(dt, aimPoint) (PlayerAnimationDriver.cs:328-355). Evalúa locomoción, torso, grafo manual, dash y aim, después aplica el recoil pendiente y al final llama a _wearableFollower.EvaluateAttachments(). Un guard por Time.frameCount impide evaluar dos veces en el mismo frame (:330-331).
3. Resuelve el aim de nuevo con el muzzle ya en su pose (WeaponManager.cs:73).
4. RefreshWeaponModes, después los ticks automáticos y al final el manual.

Motivo: los muzzles son hijos de huesos. Si el arma lee el muzzle antes de evaluar la pose, dispara desde la pose del frame anterior, y con el cuerpo girando el tiro sale desfasado. El recoil de un disparo confirmado se encola para la pose siguiente, así el muzzle no se mueve en medio de una ráfaga (`PlayerAnimationDriver.cs:706-711`).

Si `WeaponManager` sale antes de tiempo (pausa, o `GameManager` fuera de `IsPlaying`), la pose la evalúa el `LateUpdate` del driver (:357-366). El driver corre con `DefaultExecutionOrder(100)` (:12).

Limpieza y fallbacks

- `ClearEquippedWeapons` → `ClearWeapons` (`PlayerWeaponMountController.cs:112-121`): rebindea todas las armas a la mano sin sink, vacía los sets y destruye los montajes. `RemoveWeapon` (:96-110) existe, pero `WeaponManager` no lo llama.
- Sin entrada en el catálogo: `CreateMount` loguea un error y devuelve null (:179-184). El arma sí se equipa, pero conserva el binding del constructor (la mano) y dispara desde la mano aunque esté en automático.

Deuda / puntos abiertos

- `ClearWeapons` destruye los montajes junto con cualquier `PooledWeaponVfx` que tenga parentado en el muzzle. Esa entrada queda null en `WeaponVfxPool._instances`, sigue contando contra `MaxSimultaneous`, y el cue pierde el slot para siempre.
- `ClearEquippedWeapons` no llama a `CancelHeldAbilities`: un lock-on de cohete en curso queda colgado.
- `RefreshWeaponModes` corre dos veces por frame.
- El sink de aim y `_turnSpeedDegrees` no tienen efecto en wearables: es código de torreta sin uso.
- La clave por `WeaponType` impide dos armas iguales con montajes distintos. Hoy no pasa (un arma por tipo), pero no hay validación.
- El `AttachmentPath` del catálogo está vacío en las cinco entradas. Si faltara un socket en el prefab, el montaje caería en la raíz del jugador sin avisar.
- Al cambiar de manual se cancelan la ráfaga del cañón y la volea del cohete en vuelo.

PARTE 2

Enemigos y nivel

4 Catálogo de enemigos

Ficha de cada prefab de enemigo que la run puede instanciar hoy en `GameplayScene`. Los valores salen del YAML de prefabs, assets y escena. No salen del diseño. Son valores **base**. En runtime se escalan con `DifficultyManager` / `SpawnBalanceProfile` (vida $\times 6$, velocidad $\times 3$, daño $\times 1.35$ al tope, `SpawnBalanceProfile.asset:61-65`) y con los ciclos de Overheat cerrados. Ver `03-Spawning-y-enemigos.md`.

Scripts

Script	Qué aporta a la ficha
<code>EnemyHealth</code>	<code>_maxHealth</code> (<code>EnemyHealth.cs:5</code>), <code>_damageTakenMultiplier</code>
<code>SimpleFollow</code>	Velocidad de persecución <code>_speed</code> (<code>SimpleFollow.cs:8</code>). Es el follower activo en los prefabs comunes
<code>EnemyFollow</code>	<code>_moveSpeed</code> (<code>EnemyFollow.cs:10</code>). Deshabilitado (<code>m_Enabled: 0</code>) en todos los prefabs comunes; activo solo en <code>Destroyer_Boss</code>
<code>EnemyContactDamage</code>	<code>_contactDamage</code> , escalado por <code>EnemyOutgoingDamageScale</code> (<code>EnemyContactDamage.cs:21</code>)
<code>EnemyMaterialDrop</code> + <code>MaterialDropConfig</code>	Chance y pool de material (<code>EnemyMaterialDrop.cs:6-8</code>)
<code>EnemyScrapDrop</code>	Scrap: 0.1 % base en comunes, 5–10 directo al save en bosses (<code>EnemyScrapDrop.cs:7,26</code>). Se agrega en runtime (<code>SwarmPooledEnemy.cs:81</code> , <code>BossManager.cs:239</code>)
<code>EnemyDropExperience</code>	Legacy, no hace nada (<code>EnemyDropExperience.cs:3-6</code>). La XP sale del pickup de material: común +1, rara +5 (<code>MaterialCatalog.cs:54-55</code>)
<code>BossKeyDrop</code>	Instancia <code>CellBattery.prefab</code> al morir (<code>BossKeyDrop.cs:39</code>)
Behaviors	<code>FlyingRangedBehavior</code> , <code>ChargerEnemyBehavior</code> , <code>HellfireSlimeBehavior</code> , <code>BomberDroneBehavior</code> , <code>GigaWormBehavior</code> , <code>DestroyerBehavior</code>

Fuentes de spawn en `GameplayScene`

Fuente	Referencia en escena	Prefabs
<code>OrbitalSpawner</code>	<code>GameplayScene.unity:27708</code> , <code>_config = DefaultEnemySpawnRoulette</code>	Ver ruleta
<code>OverheatEliteWaveSpawner</code>	<code>GameplayScene.unity:27880</code>	<code>Slime_Elite</code> $\times 3$, <code>Drone_Elite</code> $\times 3$, <code>Chaser_Elite</code> $\times 3$ (9 por ola, ciclos impares)
<code>BossManager</code>	<code>GameplayScene.unity:32796</code>	<code>_bossPrefab = Stalker</code> , <code>_secondBossPrefab = Destroyer_Boss</code> , <code>_bossMaxHealth: 1500</code>
<code>ZoneSpawner_Slime</code> (instancia)	<code>GameplayScene.unity:27741</code> , pos (-259.7, 11.1, -179.5)	Override <code>_enemyPrefab</code> $\rightarrow$ <code>Slime</code> (variant), <code>_spawnCount: 12</code>
<code>ZoneSpawner_Drone</code> (instancia)	<code>GameplayScene.unity:27790</code> , pos (-29.7, 10.2, 73.3)	Override <code>_enemyPrefab</code> $\rightarrow$ <code>Drone</code> (variant), <code>_spawnCount: 6</code>
<code>SwarmSpawner</code>	<code>GameplayScene.unity:25411</code> , <code>m_Enabled: 0</code>	Legacy, apagado

Los `ZoneSpawner` disparan por `OnTriggerEnter` y se rearmen en cada Overheat (`_rearmOnOverheat: 1`, `ZoneSpawner.cs:99,147`). `ZoneSpawner_Chaser.prefab` no está en la escena.

Ruleta orbital (DefaultEnemySpawnRoulette.asset)

Kind	Prefab (GUID)	BaseWeight	Batch	IsVariant
0 JunkSlime	EnemyPro (e5ce45a0...)	60	4	0
1 VigilanceDrone	Drone (aefb6009...)	25	4	0
2 ChaserBot	Chaser (1780b9ab...)	25	3	0
3 HellfireSlime	Slime (variant) (f341ba25...)	0	2	1
4 BomberDrone	Drone (variant) (35733be1...)	0	2	1
5 ShockerBot	Chaser (variant) (e7399044...)	0	2	1

Bonus de variante del asset: +4 cada 120 s. **No aplica a entradas con peso 0.** BuildEffectiveEntries descarta BaseWeight <= 0 antes de sumar el bonus (EnemySpawnRoulette.cs:198 vs :203). ApplyExtraEliteChance (stat ExtraEliteChance) también sale sin hacer nada si el peso variante total es 0 (EnemySpawnRoulette.cs:231). Con el asset actual el orbital solo tira EnemyPro, Drone y Chaser (55 / 23 / 23 %), sin importar el tiempo de run ni las stats.

Fichas

Velocidad = SimpleFollow._speed, salvo que el behavior tome el movimiento. Daño = contacto / ataque del behavior. Drop = MaterialDropConfig × _dropAmount.

Comunes (orbital)

Prefab	HP	Vel.	Daño	Behavior	Drop
EnemyPro (Junk Slime)	30	2.2	contacto 12	Solo follow	JunkSlimeDrops ×1
Drone (Vigilance)	15	5 (FlyingRangedBehavior._moveSpeed)	contacto 10; bala 15, vel 12	Vuela a altura 4, se acerca hasta 15 m, lockea 2 s, dispara, cooldown 1.5 s	VigilanceDroneDrops ×1
Chaser	50	3	contacto 18; embestida 25	A 6 m carga 2 s y dashea (30 u/s, máx. 12 m); si no pega en 2 s, overheatea	ChaserBotDrops ×1

Variantes (solo por ZoneSpawner, o nunca)

Prefab	HP	Vel.	Daño	Behavior	Drop	¿Spawnea?
Slime (variant) (Hellfire)	60	5	contacto 15; explosión 25 (r 3.5), burn 2 dps × 3 s, FireArea 4/tick c/0.5 s × 4 s	A 4 m se vuelve invencible, se lanza, explota y muere	HellfireSlimeDrops ×1	Sí, zona Slime (12)
Drone (variant) (Bomber)	25	5.5	contacto 10; bala 20, vel 15; C4 20 (EnemyC4.prefab)	A 10 m hace 2 dashes con 2 C4 cada uno, recarga 2 s, entre tanto dispara	BomberDroneDrops ×1	Sí, zona Drone (6)
Chaser (variant) (Shocker)	100	3.5	contacto 20; embestida 12 + descarga 10, stun 1 s	Charger con _overcharged: 1, dash 38 u/s, máx. 14 m	ShockerBotDrops ×1	No. Solo en Testing/enemiesTesting.unity

Elites (oleada de Overheat impar)

Prefab	HP	Vel.	Daño	Behavior	Drop
Slime_Elite	450	5	contacto 25; explosión 25, burn 4 dps	Hellfire (muere al explotar)	JunkSlimeDrops ×2
Drone_Elite	125	5.5	contacto 10; bala 35, vel 8; C4 20	Bomber, 4 C4 por dash, recarga 5 s	VigilanceDroneDrops ×2
Chaser_Elite	300	3.5	contacto 35; embestida 12 + descarga 10	Shocker (_overcharged: 1), empuje 50	ChaserBotDrops ×2

El modo Shocker sí está en juego, pero vía Chaser_Elite. El daño de embestida es `_hitDamage + _dischargeDamage` (ChargerEnemyBehavior.cs:179).

Bosses (Overheat par)

Prefab	HP prefab → spawn	Vel.	Daño	Behavior
Stalker (GigaWormBehavior)	1500 → 1500. _damageTakenMultiplier: 0.2; EnemyDamageHitZone ×1.2	5 bajo tierra; salto 25, caída 25	contacto 60; emerge 80 (r 14); escupitajo → CorrosiveSlimeArea 2/tick c/0.5 s × 5 s, slow 0.45	Caza bajo tierra, telegraph 1.2 s, salta (apex 50), cae donde estaba el jugador, escupe 2 proyectiles c/0.5 s
Destroyer_Boss (DestroyerBehavior)	1000 → 1500 . Punto débil de boca 80 HP	1.6 (EnemyFollow)	contacto 10; misil 12 c/1.2 s; tragar 40	Hunt con misiles guiados; succión al 75/50/25 %: inmune, cura 2 % por enemigo comido y 15 % si traga al jugador; romper el punto débil le saca 20 %; corte a los 15 s

BossManager.cs:237 pisa la vida del prefab con `_bossMaxHealth`. Orden (BossManager.cs:313): ciclo 2 → Stalker, 4 → Destroyer, 6 → Stalker... Ninguno de los dos tiene EnemyMaterialDrop. Sueltan: llave CellBattery (BossKeyDrop en prefab), 5–10 de scrap al save y powerup temporal (los dos últimos, agregados en BossManager.cs:239-242).

Drop tables (assets en ScriptableObject/Economy/Drops/)

Asset		Pool
JunkSlimeDrops	0.65	Chapa 71, Caños 12, Engranajes 12, Combustible gelificado 5
VigilanceDroneDrops	0.65	Engranajes 95, Explosivo plástico 5
ChaserBotDrops	0.75	Caños 95, Cableado 5
HellfireSlimeDrops	0.9	Chapa 40, Combustible 60
BomberDroneDrops	0.9	Engranajes 40, Explosivo 60
ShockerBotDrops	0.9	Caños 40, Cableado 60 (solo lo usa el Shocker, que no spawnear)

CSV y BalanceAutoImporter

- Los CSV (Assets/Data/Balance/balance_material_usage.csv, balance_weapon_stats.csv) no tienen datos de enemigos. No hay CSV de HP, velocidad ni daño; esos valores viven solo en los prefabs.
- BalanceAutoImporter corre en cada domain reload, pero sale temprano si existe MaterialUsageBalance.asset (BalanceAutoImporter.cs:25). Hoy existe, así que no importa nada. Esto contradice la nota de memoria de que "cada reload pisa assets", al menos en el HEAD actual.

3. Si ese asset falta, además llama `CreateDefaultDropConfigs` (`BalanceImportMenu.cs:31-46`), que **reescribe** los 6 drop configs con valores hardcodedos distintos a los del repo. Por ejemplo, Junk 0.45 con 95/5 (hoy 0.65, 4 materiales), Chaser 0.55 (hoy 0.75), variantes 0.55 (hoy 0.9).

Deuda / puntos abiertos

- **Variantes con peso 0 no salen nunca del orbital.** El bonus de +4/120 s está muerto porque se descartan antes (`EnemySpawnRoulette.cs:198`). Pasa lo mismo con `ExtraEliteChance`. `BALANCE_NOTES.md` todavía describe el bonus como activo (+3).
- **Chaser (variant) / ShockerBot sin uso** en `GameplayScene`: ruleta con peso 0 y sin `ZoneSpawner`. Tampoco se usa `ShockerBotDrops`.
- **03-Spawning-y-enemigos.md desactualizado.** Dice que no hay `ZoneSpawner` en la escena (hay 2, con Hellfire y Bomber) y que el `BossManager` usa `Boss/Boss_2` (usa `Stalker` y `Destroyer_Boss`).
- **Scavenging con escala mezclada.** `EnemyMaterialDrop.cs:29` hace `chance × (1 + Scavenging)`, con Scavenging base 50 (`Scavenging.asset:17`) → chance siempre 1. `EnemyScrapDrop` y `PlayerDropMath` lo tratan como porcentaje ($\div 50$, $\div 100$). Si `GetStat` devuelve 50, `_dropChance` de los configs no tiene efecto. No se verificó en play.
- **Elites con drop de su familia base.** `Slime_Elite` usa `JunkSlimeDrops`, NO `HellfireSlimeDrops`, aunque se comporta como Hellfire. Lo mismo `Drone_Elite` y `Chaser_Elite`.
- **Destroyer 1000 HP en prefab, 1500 en run.** El valor del prefab engaña si se lo testea suelto.
- **Comentario viejo en BomberDroneBehavior.cs:3-6.** Dice 3 dashes × 5 C4; los prefabs usan 2×2 (variante) y 2×4 (elite).
- **BossKeyDrop depende del prefab.** Un boss nuevo sin el componente no suelta llave.
- **CreateDefaultDropConfigs pisaría los drops afinados** si se borra `MaterialUsageBalance.asset` o se usa el menú.

5 Nivel y entorno

Qué hay en `GameplayScene` hoy y qué scripts arman el objetivo de salida, los pickups de mundo y el entorno visual. El nivel sigue siendo un greybox (`Map`, cubos y planos) con una sola estación de crafeo y una puerta de salida tipo compactador. La salida se abre con llaves que dropean los bosses. Ver `04-Game-Loop.md` para la fase de escape y `03-Spawning-y-enemigos.md` para la presión de spawn.

Scripts

Script	Ubicación	Rol
<code>LevelExitObjective</code>	<code>Level/</code>	Cuenta llaves. Singleton <code>Instance</code> , <code>[DefaultExecutionOrder(-40)]</code>
<code>KeyPickup</code>	<code>Level/</code>	<code>IPickable</code> : registra la llave y se destruye
<code>WorldPickup</code>	<code>Level/</code>	Caída al suelo, bobbing, imán y recogida por radio
<code>BossKeyDrop</code>	<code>Level/</code>	En la muerte del boss instancia la llave (<code>CellBattery</code>)
<code>ExitDoor</code>	<code>Level/</code>	Máquina de estados de la puerta, input [E], victoria
<code>CompactorDoorPresentation</code>	<code>Level/</code>	Muestra el clip de apertura según la carga de la puerta
<code>LevelExitPressure</code>	<code>Level/</code>	Con todas las llaves: Overheat permanente y escalones $\times 2/\times 3/\times 4$
<code>ExitSpawnPressure</code>	<code>Level/</code>	Estático. Pasa el multiplicador a <code>OverheatSwarmBoost</code>
<code>LevelExitHud</code>	<code>Level/</code>	Texto de estado (llaves, carga, presión). Vive en <code>GameplayHudV2.prefab</code>
<code>GuideArrow / GuideArrowController</code>	<code>Level/</code>	Flecha sobre el jugador hacia la estación y luego hacia la puerta
<code>BelowLevelFogFade</code>	<code>Level/</code>	Plano de niebla falsa con fade según la altura del jugador
<code>PickupGroundFall</code>	<code>raíz de Assets/Scripts/</code>	Caída por gravedad sin <code>Rigidbody</code> para drops
<code>SpawnGroundUtility</code>	<code>raíz de Assets/Scripts/</code>	Snap al suelo + depenetración de un <code>CharacterController</code> al spawnear
<code>CraftingStation</code>	<code>Economy/</code>	Abre <code>CraftingUI</code> con [E] dentro del radio

`PickupGroundFall` y `SpawnGroundUtility` no están en `Level/` aunque son utilidades de entorno. Tools de editor en `Level/Editor/` (`LevelExitSetupEditor`, `CompactorEscapeSetupEditor`, `GuideArrowSetupMenu`, `BelowLevelFogPrefabBuilder`, `GameplayModelsSetupEditor`): se documentan aparte.

Objetivo de salida

- Cada boss con `BossKeyDrop` instancia la llave en `posición + 0.75 Y` al morir, salvo que ya estén todas (`BossKeyDrop.cs:26-39`). No hace raycast: la caída la resuelve `WorldPickup`.
- `WorldPickup` cae con `PickupGroundFall` y, al aterrizar, pasa a bobbing + imán (`WorldPickup.cs:102-139`). Radio base 1.5, imán 5, velocidad 10; el radio efectivo sale de `PlayerStatMath.GetPickupRange` (`WorldPickup.cs:150-157`).
- `KeyPickup.OnPickedUp` → `LevelExitObjective.RegisterKey()` (`KeyPickup.cs:13`). En escena `_keysRequired: 2` (`GameplayScene.unity:34379`).

4. Con la última llave dispara `OnAllKeysCollected` (`LevelExitObjective.cs:36-48`). Escuchan la puerta, la presión, el HUD y la flecha.
5. `LevelExitPressure.ActivatePressure: BossManager.SetExitPhaseActive(true)`, elites deshabilitados, `EnterPermanentOverheat()` y `ExitSpawnPressure.SetActive(true, 2)` (`LevelExitPressure.cs:78-96`).
6. Cada `_minutesPerTier` (1 min) sube el escalón hasta el último del array `{2, 3, 4}` (`LevelExitPressure.cs:58-76`; escena :34428-34432).

Puerta de salida

Estados (`ExitDoor.cs:7-19`): `Locked` → `AwaitingActivation` → `Charging` → `Ready` → `Used`.

Paso	Qué pasa	Código
Llaves completas	<code>Locked</code> → <code>AwaitingActivation</code> , <code>OnDoorUnlocked</code>	<code>ExitDoor.cs:122-129</code>
[E] en radio	<code>TryStartCharging</code> , <code>OnChargeStarted</code>	<code>ExitDoor.cs:111-113</code> , <code>132-146</code>
Carga	Cuenta regresiva, <code>OnChargeProgress</code> cada frame	<code>ExitDoor.cs:158-172</code>
Fin de carga	<code>Ready</code> , <code>OnDoorReady</code>	<code>ExitDoor.cs:169-171</code>
[E] otra vez	<code>Used</code> → <code>GameManager.TriggerVictory()</code>	<code>ExitDoor.cs:115-118</code>

- No reacciona con `CraftingUI` o `LevelUpChoiceUI` abiertas (`ExitDoor.cs:90-91`).
- La distancia es solo XZ (`flat.y = 0`, `ExitDoor.cs:103-106`).
- Valores: el prefab trae 5 s de carga y radio 3 (`CompactorExitDoor.prefab:214-215`). La escena los pisa a **radio 20 y 20 s** (`GameplayScene.unity:12892-12899`).
- `ResetCharge()` vuelve a `AwaitingActivation` o `Locked` sin perder llaves (`ExitDoor.cs:149-156`).

`CompactorDoorPresentation` arma un `PlayableGraph` manual (sin `Animator Controller`) y hace `SetTime(progreso × largo del clip)` (`CompactorDoorPresentation.cs:140-144`, `146-170`). El collider de puerta cerrada se apaga en `Ready/Used`; el `_Activity` del renderer de succión va de 0.2 a 1 durante la carga (:130-133). Resincroniza en `LateUpdate`.

Crafteo y flecha guía

- Hay **una** `CraftingStation` en escena (`GameplayScene.unity:30110`), en (-163.5, 56.51, 68.85) (:30143), con el `Workbench.fbx` como hijo y un `Workbench Collision`. Radio de interacción pisado a 10 (:30128; default 3 en `CraftingStation.cs:11`).
- La estación mide distancia 3D (`CraftingStation.cs:50`); la puerta, solo XZ.
- No abre si otra UI de crafteo o de level-up está visible (`CraftingStation.cs:42-43`). Al abrir dispara `OnInteracted` (:62).
- `GuideArrowController` (sobre la instancia de `Arrow_Guide`, `GameplayScene.unity:25737`): a los 20 s de run apunta a la estación; con todas las llaves apunta a la puerta. Cada aparición dura 5 s o hasta interactuar (`OnInteracted / OnChargeStarted`) (`GuideArrowController.cs:58-93`; escena :25825-25826).
- `GuideArrow` flota 2.2 sobre el jugador y apunta en 3D con pulso de escala (`GuideArrow.cs:15`, 64-88).

Utilidades de suelo

`PickupGroundFall.Tick` (`PickupGroundFall.cs:20-36`): gravedad -20, raycast desde +5 con alcance 85, ignora triggers. Lo usan `WorldPickup.cs:105`, `XPDrop.cs:64` y `MaterialDrop.cs:129`.

`SpawnGroundUtility.TryResolveFootPosition` (`SpawnGroundUtility.cs:18-71`):

1. Raycast hacia abajo en XZ. Entre todos los hits elige el más cercano a la Y de referencia, con umbral opcional. Si ninguno entra en el umbral, igual toma el más cercano (:73-130).
2. Si la máscara primaria falla, prueba la de fallback (:43-49).
3. Hasta N iteraciones de depenetración de la cápsula del `CharacterController` (:54-67).

La usan `OrbitalSpawnPlacement.cs:141`, `BossManager.cs:208`, `SwarmSpawner.cs:162` y `BalanceTestingController.cs:363`.

GameplayScene: layout

Objetos raíz (transform sin padre), agrupados:

Grupo	Objetos
Managers	<code>GameManager</code> , <code>DifficultyManager</code> , <code>OverheatSystem</code> (<code>HeatManager</code> + <code>BossManager</code>), <code>AudioManager</code> , <code>BalanceTuning</code> , <code>LevelExitObjective</code> , <code>LevelExitPressure</code>
Spawn	<code>OrbitalSpawner</code> , <code>EliteWaveSpawner</code> , <code>SpawnPoint</code> (<code>SwarmSpawner</code> deshabilitado, <code>m_Enabled: 0</code> , :25409), <code>SwarmEnemyPool</code>
Pools	<code>GameplayPools</code> , <code>XP Pool</code> , <code>MaterialPool</code> , <code>ProjectilePool</code> (prefab)
Nivel	<code>Map</code> (greybox, escala 15, 27 hijos <code>Cube/Plane</code> , :30839), <code>CraftingStation</code> , <code>ExitDoor</code> (prefab), <code>BelowLevelFogPlane</code> (prefab), <code>Lights</code> , <code>Global Volume</code>
Jugador / cámara	<code>player</code> (prefab), <code>Main Camera</code> (<code>ThirdPersonCamera</code>), <code>Arrow_Guide</code> (prefab)
UI	UI (14 hijos: HUD V2, menús de run, <code>EventSystem</code> , canvases de stats/inventario/logros)
Debug	<code>DebugUI</code> , <code>DEBUGTESTINVENTORY</code> (<code>DebugCrafting</code>), <code>DEBUG_InfiniteHealth</code>
Sueltos	<code>EnemyPro</code> (prefab, activo, :10807), <code>Enemy</code> (prefab, inactivo, :30689)

Posiciones clave:

Qué	Posición	Línea
Spawn del jugador (<code>player.prefab</code>)	(-150.9, 0.3, -50.7)	:34695
<code>ExitDoor</code> (<code>CompactorExitDoor.prefab</code> , escala 15.04)	(-169.8, -4.2, -243.4)	:12913
<code>CraftingStation</code>	(-163.5, 56.51, 68.85)	:30143
<code>ZoneSpawner_Slime</code> → <code>Slime</code> (variant)	(-259.7, 11.1, -179.5)	:27751
<code>ZoneSpawner_Drone</code> → <code>Drone</code> (variant)	(-29.7, 10.2, 73.3)	:27800
<code>BelowLevelFogPlane</code> (300 × 300, rotado 90° en X)	(-121, 38.4, -35.5)	:29314

Los dos `ZoneSpawner` son instancias de prefab activas: 12 enemigos, radio 6, `_rearmOnOverheat: 1` (`ZoneSpawner_Slime.prefab:70-76`). En la escena se pisa `_enemyPrefab` y `_groundRaycastMask` (bits 129).

Niebla de piso

`BelowLevelFogFade` pone `_Visibility` por `MaterialPropertyBlock: 1` si el jugador está por encima de `planoY - 0.35`, 0 si está por debajo, con fade de 0.75 s. En 0 apaga el renderer (`BelowLevelFogFade.cs:71-105`). Crea o ajusta un `BoxCollider` trigger de 4 unidades de grosor que solo mira al jugador (:107-121). La niebla de `RenderSettings` está apagada; este plano es la única niebla.

Iluminación y URP

Escena (GameplayScene.unity:17-97):

- Fog de Unity off (`m_Fog: 0`). Ambient desde el skybox (`m_AmbientMode: 0`), skybox por defecto (Default-Skybox, fileID 10304), sin `m_Sun`.
- Lightmaps horneados on, realtime GI off, Mixed = Shadowmask. `LightingSettings = Assets/New Lighting Settings.lighting` (bake res 40, atlas 1024, sin AO). Hay `LightingDataAsset` horneado.
- No hay luz direccional.** `Lights` agrupa 4 luces **Point** (aunque se llaman "Spot Light"), naranjas (`1, 0.58, 0`), intensidad 1000, rango 49.4, sombras hard, modo **Baked** (`m_Lightmapping: 2`) (`:3255, :3943, :15042, :23873`).
- Global Volume: global, prioridad 0, peso 1, perfil `SampleSceneProfile (:13219-13223)`.

SampleSceneProfile: Bloom (threshold 1, intensidad 0.25, scatter 0.5), Vignette 0.2, Tonemapping Neutral, MotionBlur desactivado. El mismo perfil está también como volume por defecto del RP asset de PC (`PC_RPAsset.asset:95`).

Pipeline: `GraphicsSettings` usa `PC_RPAsset` (`GraphicsSettings.asset:43`). Hay dos calidades: Mobile → `Mobile_RPAsset` y PC → `PC_RPAsset`. La actual es PC (`m_CurrentQuality: 1`); Standalone usa PC y Android usa Mobile.

Setting	PC	Mobile
Render scale	1	0.8
MSAA	off (1x)	off (1x)
HDR	on	on
Shadow distance	50	50
Cascadas / resolución main light	4 / 2048	1 / 1024
Luces adicionales	per-pixel, 4 por objeto, con sombras	per-pixel
Rendering path	Forward+ (<code>PC_Renderer.asset:56</code>)	Forward
Renderer features	SSAO (intensidad 0.4, radio 0.3, fuente depth-normals)	ninguna

Deuda / puntos abiertos

- ZoneSpawner sí está en la escena.** `03-Spawning-y-enemigos.md` dice que no hay instancias, pero hay dos activas (Slime y Drone, 12 cada una, rearme en Overheat). Hay que corregir ese doc o sacarlas.
- No hay luz direccional ni `m_Sun`.** El RP de PC reserva sombras de main light (4 cascadas, 2048) que no tienen luz que las use. La iluminación son 4 point lights horneadas: nada dinámico proyecta sombra real salvo lo que haga SSAO.
- La escena pisa la puerta a radio 20 y 20 s de carga.** El prefab trae 3 y 5 s. Como el chequeo ignora Y (`ExitDoor.cs:103-106`), con radio 20 se puede activar desde otro nivel de altura sobre o bajo la puerta.
- GuideArrowController resuelve una sola estación** con `FindAnyObjectByType<CraftingStation>` (`GuideArrowController.cs:35`), aunque `CraftingStation` ya contempla que haya varias (`CraftingStation.cs:40-41`). Hoy hay una; si se agregan más, la flecha apunta a una arbitraria.
- Hay objetos sueltos en la escena:** `EnemyPro` activo en la raíz en (8.7, 0.6, -13.8), `Enemy` inactivo, `SpawnPoint` con el `SwarmSpawner` obsoleto deshabilitado y `SwarmEnemyPool` activo.
- Nombres de plantilla.** El volume usa `SampleSceneProfile`, los lighting settings son `New Lighting Settings`, y las luces "Spot Light" en realidad son point lights.
- El trigger de `BelowLevelFogFade` sobra:** `Update` ya recalcula el target cada frame (`BelowLevelFogFade.cs:44-54`). `OnTriggerEnter/Exit` solo repiten ese cálculo.
- PickupGroundFall y SpawnGroundUtility están en la raíz de `Assets/Scripts/`,** fuera de `Level/`.

PARTE 3

Economía y progresión

6 Economía de materiales y balance CSV

Cómo se generan, se recogen y se gastan los materiales dentro de la run, y de dónde salen los números de balance (CSV, ScriptableObjects y el hub de spawn). Los valores serializados en assets y prefabs mandan sobre los defaults de script. Cuando difieren, van los dos.

Scripts

- `Economy/EconomyBootstrap.cs`: singleton `DontDestroyOnLoad` que se autocrea antes de cargar la escena. Carga el balance de materiales y reimporta el CSV de armas.
- `Economy/CsvReader.cs`: parser CSV propio (comillas, "", saltos de línea dentro de comillas). Lee con UTF-8 y descarta el BOM.
- `Economy/MaterialUsageParser.cs`, `Economy/WeaponStatsParser.cs`: traducen los CSV a datos.
- `Economy/MaterialCatalog.cs`: categoría (común o rara), nombres, colores y XP de cada material.
- `Economy/MaterialDropConfig.cs` + `Economy/EnemyMaterialDrop.cs`: tirada de drop por enemigo.
- `Economy/MaterialPool.cs`, `MaterialDrop.cs`, `MaterialPickupReceiver.cs`, `MaterialInventory.cs`: orbe, recogida e inventario.
- `Economy/WeaponCraftingService.cs` + `WeaponCraftingCostCalculator.cs`: mejora, Tinkering y Advanced Tinkering.
- `Economy/MaterialUsageBalanceSO.cs`: roles por columna de arma y totales por nivel.
- Editor: `Economy/Editor/BalanceImportMenu.cs`, `BalanceAutoImporter.cs`, `MaterialUsageCsvImporter.cs`, `WeaponStatsCsvImporter.cs`, `Weapon/Editor/WeaponBalanceSheetApplier.cs`.
- `Balance/BalanceTuningHub.cs` + `Balance/SpawnBalanceProfile.cs` (+ `Balance/Editor/BalanceTuningHubEditor.cs`).

Materiales

Material	Categoría	XP al recoger
SheetMetal, MetalPipe, Gears	Común	1
JellifiedFuel, PlasticExplosive, Wiring	Rara	5

`MaterialCatalog.cs`:6-14 y :54-55. La XP es `GetPickupXpValue × amount` (`MaterialPickupReceiver.cs`:39-43), así que un doble drop también duplica la XP. El doc 05 cita el DEV del 21-7 (común = 4, rara = 12). El código da **1 y 5**.

Drop de materiales

Flujo en `EnemyMaterialDrop.OnEnemyDied` (`EnemyMaterialDrop.cs`:19-54):

1. `chance = DropChance × (1 + Scavenging)`, con clamp a 0-1 (:27-29).
2. `Random.value > chance`: no hay drop.
3. `MaterialDropConfig.TryRoll`: ruleta por `WeightPercent`, acumulada sobre 100 (`MaterialDropConfig.cs`:21-46).
4. `amount = _dropAmount`. Se duplica si `Random.value < clamp01(DoubleDrop)` (:56-61).
5. `MaterialPool.TrySpawn`. El pool de `GameplayScene` serializa 128 iniciales, con crecimiento hasta **612** (el default de script es 2048).

Configuración serializada (`Assets/ScriptableObjects/Economy/Drops/`):

Asset	DropChance	Pool	Prefabs ()
JunkSlimeDrops	0.65	SheetMetal 71 / MetalPipe 12 / Gears 12 / JellifiedFuel 5	EnemyPro (1), Slime_Elite (2)
HellfireSlimeDrops	0.9	SheetMetal 40 / JellifiedFuel 60	Slime (variant) (1)
ChaserBotDrops	0.75	MetalPipe 95 / Wiring 5	Chaser (1), Chaser_Elite (2)
ShockerBotDrops	0.9	MetalPipe 40 / Wiring 60	Chaser (variant) (1)
VigilanceDroneDrops	0.65	Gears 95 / PlasticExplosive 5	Drone (1), Drone_Elite (2)
BomberDroneDrops	0.9	Gears 40 / PlasticExplosive 60	Drone (variant) (1)

`BalanceImportMenu.CreateDefaultDropConfigs` (`BalanceImportMenu.cs:30-48`) escribe otros valores (0.45 / 0.65 / 0.55 y 95/5 o 40/60). Si se ejecuta, pisa los assets de arriba.

Inconsistencia de Scavenging

Hay dos fórmulas para el mismo stat:

Dónde	Drop	Doble drop
<code>EnemyMaterialDrop.cs:29</code> / <code>:60</code> (runtime real)	<code>DropChance × (1 + Scavenging)</code>	<code>clamp01(DoubleDrop)</code>
<code>PlayerStatMath.cs:73</code> / <code>:86</code> (<code>PlayerDropMath</code> , solo <code>PassiveItemTestingController</code>)	<code>Scavenging / 100</code>	<code>(DoubleDrop + Scavenging - base) / 100</code>

`Scavenging.asset` tiene base **50** y `DoubleDrop.asset` tiene base **-30**. Con la fórmula del runtime, la chance es `clamp01(0.65 × 51) = 1`: **todo kill dropea**, y la `DropChance` de los assets no tiene efecto. El doble drop arranca en 0, y cualquier bonus positivo neto lo lleva directo a 100 %. La herramienta de sandbox muestra 50 % y otra chance de doble drop. Queda confirmado: las dos fórmulas no coinciden.

Crafting

`WeaponCraftingService` vive en el prefab `player.prefab`. Serializa `_materialBalance = MaterialUsageBalance.asset` y un `_weaponPool` de 5 armas.

- **Mejora** (`TryUpgradeWeapon`, `WeaponCraftingService.cs:68-90`): hasta nivel 10. El paso 5→6 se bloquea sin path (`:79-80`). El coste sale de `GetUpgradeCost(targetLevel)`. `CraftingUI` siempre pide `Level + 1` (`CraftingUI.cs:262`).
- **Tinkering** (`:92-112`): se desbloquea un arma al azar del pool que no esté equipada y que esté desbloqueada en el save.
- **Advanced Tinkering** (`:149-186`): solo en nivel 5. Ofrece `PathA`, o `PathB` si está desbloqueado (50/50). La oferta se guarda y reabrir la estación no la vuelve a tirar (`:122-130`). Rechazar la oferta cobra y garantiza el path alternativo. Aceptarla sube a 6 y aplica el path.

Coste de mejora (`WeaponCraftingCostCalculator.cs:27-51`): para cada material con rol en la columna del arma (base o `FlameA...BladesB` desde nivel 6, `MaterialUsageBalanceS0.cs:61-82`), se toma el total del rol en ese nivel.

Nivel destino	Principal (X)	Secundario (x)	Terciario (y)	Principal extra (XX)
2	15	0	0	0
3	25	10	0	0
4	50	20	0	0
5	100	40	0	0

Nivel destino	Principal (X)	Secundario (x)	Terciario (y)	Principal extra (XX)
6	— (Advanced Tinkering)	—	—	—
7	100	50	10	20
8	125	60	20	40
9	150	80	40	80
10	225	120	100	275

Valores tomados de `MaterialUsageBalance.asset (_roleTotals)`, que coinciden con el CSV. La columna "Base" del CSV se importa como nivel 1 (15/10/10/10), pero ninguna acción la usa.

Roles (CSV, 39 asignaciones):

Columna	Principal	Secundario	Terciario / extra
Flamethrower	Gears	MetalPipe	—
Rocket launcher	MetalPipe	SheetMetal	—
Mortar	MetalPipe	Gears	—
Autocannon	Gears	SheetMetal	—
Rotating Blades	SheetMetal	Gears	—
Flame A / B	Gears	MetalPipe	y: JellifiedFuel / Wiring
Rocket A / B	MetalPipe	SheetMetal	y: PlasticExplosive / Gears
Mortar A / B	MetalPipe	Gears	y: SheetMetal / PlasticExplosive
Auto A / B	Gears	SheetMetal	y: JellifiedFuel / MetalPipe
Blades A	— (SheetMetal es XX)	Gears	XX: SheetMetal
Blades B	SheetMetal	Gears	y: Wiring

Costes fijos (`WeaponCraftingCostCalculator.cs:53-93`):

Acción	Slot 1	Slot 2	Slot 3
Tinkering (cada común)	gratis	5	15
Advanced Tinkering (cada rara)	5	15	30
Advanced tras rechazo	8	22	45

El "slot" de Advanced es la cantidad de armas que ya tienen path + 1 (`WeaponCraftingService.cs:222-237`).

Pipeline CSV

CSV	Qué alimenta	Estado
<code>balance_material_usage.csv</code>	Roles X/x/y/XX por columna de arma y filas "Total ..." por nivel → <code>MaterialUsageBalance.asset</code>	Funciona
<code>balance_weapon_stats.csv</code>	Bloques por arma: filas Total/Added de Damage, rangos, munición, habilidad, crítico, knockback, columna "Scales with" y stats específicas de path	No importa nada (ver deuda)

Entradas:

1. Menú ScrapWaves/Balance/Import All CSV (BalanceImportMenu.cs:13-28): reescribe MaterialUsageBalance.asset y los 5 WeaponSO.
2. BalanceAutoImporter (BalanceAutoImporter.cs:17-30): en cada carga del editor, **solo si** MaterialUsageBalance.asset **no existe**. En ese caso llama a ImportAll y a CreateDefaultDropConfigs. El asset existe, así que hoy no dispara. La nota de memoria "reimporta en cada domain reload" ya no aplica a este script.
3. EconomyBootstrap.LoadBalanceData (EconomyBootstrap.cs:41-62): en cada Play, si el SO no tiene roles, parsea el CSV de materiales. **Siempre** corre WeaponStatsParser.ImportAll sobre los WeaponSO reales (cargados por AssetDatabase, :76-84). En build, las rutas Assets/... no existen y no se importa nada. WireCraftingServices corre antes de cargar la escena y no encuentra servicios, así que queda la referencia serializada del prefab.
4. WeaponBalanceSheetApplier (menú Tools/ScrapWaves/Apply Weapon Balance Sheet Snapshot): snapshot hardcodeado (WeaponBalanceSheetApplier.cs:36-111) que escribe BaseDamage, munición, cooldown y LevelData base y de paths.

Balance de spawn

BalanceTuningHub (GameplayScene, orden -60) inyecta Assets/Data/Balance/SpawnBalanceProfile.asset en DifficultyManager, HeatManager y OrbitalSpawner, solo en los que no tienen profile propio (BalanceTuningHub.cs:47-58). También expone lecturas en vivo y dos acciones de debug: fijar el heat por % de barra y forzar el Overheat (:131-159).

Campo	Asset	Default script
Delay antes del escalado (s)	30	30
Curva intensidad (min → valor)	0→0, 5→0.2, 10→0.5, 20→1	0, 5→0.21, 10→0.40, 19.46→1
Velocidad de rampa	0.95	1
Máx. multiplicador de cantidad (tiempo)	3	4
Máx. vida / velocidad / daño enemigo	6 / 3 / 1.35	10 / 4 / 1.35
Cantidad a heat lleno / intervalo a heat lleno	2.5 / 0.5	2 / 0.45
Ciclo: paso y piso de intervalo	0.25 / 0.6	0.1 / 0.3
Ciclo: paso y techo de batch y de vida	0.25 / 2	0.1 / 2
Heat: 0→80 % / 80→100 % (puntos)	60 / 60	100 / 100
Escalada por Overheat	1.4	1.5
Heat por kill / decay post-Overheat	1 / 25	5 / 12
Intervalo base spawner (s) / máx. activos	15 / 300	5 / 300

Deuda / puntos abiertos

- **El CSV de armas no se parsea.** WeaponStatsParser.ApplyBlock busca "Basic" en la columna 1 (WeaponStatsParser.cs:80) y "Total" en la columna 0 (:120), pero el CSV los tiene en las columnas 0 y 1. Entonces vacía BalanceStats y UpgradeSpecificStats y sale. Después SyncLegacyLevelData (:208-230) reescribe LevelData 1-10 con multiplicador 1. Como EconomyBootstrap lo corre en cada Play sobre los assets reales, los WeaponSO commiteados tienen BalanceStats: [] y LevelData base plano en 1.0. Los niveles 1-5 no escalan daño. Los LevelData de path conservan el snapshot.
- BaseDamage commiteado (Flamethrower 5, RocketLauncher 25, Mortar 40) no coincide ni con el CSV (25/70/...) ni con el snapshot del applier. Hay que decidir una sola fuente.
- ApplyDerivedBaseStats asigna ActiveAbilityAmmoCost desde "Ability damage" (WeaponStatsParser.cs:203). Hoy está muerto, pero es un bug latente.

- Las dos fórmulas de Scavenging y DoubleDrop (arriba) no coinciden. Con la del runtime, la DropChance de los assets de drop no tiene efecto.
- TryTinkerRandomWeapon cobra antes de comprobar que haya candidatos (WeaponCraftingService.cs:101-107). Si no quedan armas desbloqueadas, se pierden los materiales.
- El mensaje de rechazo dice "+50 %", pero los costes reales son +60 % / +47 % / +50 % (WeaponCraftingService.cs:185 vs WeaponCraftingCostCalculator.cs:77-82). El parámetro advancedRejected de GetTinkeringSlotCost no se usa.
- CreateDefaultDropConfigs tiene valores viejos y pisa los drops si se regenera el asset de balance.
- MaterialCatalog da 1/5 de XP, contra el 4/12 del DEV citado en el doc 05.

7 Power-ups temporales y Scrap

Cubre los power-ups que caen durante la run (buffs cortos, curación, nuke) y el Scrap, la moneda meta que se gana en la run y se gasta fuera de ella. Los números de los power-ups escalan con el nivel del jugador. El Scrap entra por tres vías: pickups raros, kills de boss y liquidación al final de la run.

Scripts

- `Powerups/TemporaryPowerupController.cs`: aplica los efectos en el jugador. `SpecMetaBootstrap` lo agrega en runtime si falta (`SpecMetaBootstrap.cs:91-92`). Solo está serializado en `Scenes/Testing/test_balance.unity`.
- `Powerups/EnemyTemporaryPowerupDrop.cs`: tirada de power-up al morir un enemigo.
- `Powerups/TemporaryPowerupPickup.cs`, `TemporaryPowerupPool.cs`, `TemporaryPowerupVisualCatalog.cs`: pickup, pool y visuales.
- `Powerups/EnemyScrapDrop.cs`, `ScrapPickup.cs`, `ScrapPickupPool.cs`: Scrap dentro de la run.
- `GameManager.cs` (`CalculateScrapEarned`, `ReportRunToSaveSystem`): liquidación al final de la run.
- `Enemy/BossManager.cs`, `Enemy/SwarmPooledEnemy.cs`: agregan los componentes de drop en runtime.

Ningún prefab de enemigo tiene serializados `EnemyScrapDrop` ni `EnemyTemporaryPowerupDrop`. Se agregan en runtime: `SwarmPooledEnemy.EnsureMetaDropComponents` (`SwarmPooledEnemy.cs:77-83`) para los enemigos del pool y `BossManager` (`BossManager.cs:239-242`) para cada boss.

Drop de power-ups

`EnemyTemporaryPowerupDrop.OnEnemyDied` (`EnemyTemporaryPowerupDrop.cs:32-45`):

1. Base 1 % para normales y 5 % si `WeaponEnemyClassifier.CountsAsEliteOrBoss` da verdadero (:9-10). Elite incluye los nombres con "Elite" o "variant".
2. `chance = base × (Scavenging / 50)`, con clamp a 0–1 (:39). Con el Scavenging base (50), la chance queda en 1 % y 5 %.
3. El tipo es uniforme entre los 6 (`Random.Range(0, 6)`, :43). No hay pesos.
4. `TemporaryPowerupPool.TrySpawn`. El pickup se recoge caminando: `WorldPickup.ConfigureForGameplayCollection(1.5, 6, 12)` (`TemporaryPowerupPickup.cs:33`).

El pool se autocrea (`TemporaryPowerupPool.cs:60-71`), con 8 iniciales y 48 como máximo. Carga los prefabs `Assets/Prefabs/Pickups/powerups/PowerUp_*.prefab` **solo en editor**, vía `AssetDatabase` (:214-229). En build no hay referencias serializadas, así que se usa la esfera de fallback, tinteada por tipo, con la malla de `Resources/TemporaryPowerupVisualCatalog.asset`.

Efectos

$t = \text{clamp}_{01}(\text{nivel} / \text{LevelCap})$, con `LevelCap` 36 (`TemporaryPowerupController.cs:289-294`). Los rangos van de `Lerp(min, max, t)`: en nivel 1, $t \approx 0.03$; en nivel 36, $t = 1$.

Tipo	Duración	Efecto ()
ExtraDamage	15 s	<code>DamageMultiplier</code> ×1.5–2.5, <code>AttackSpeedMultiplier</code> ×1.75–3, CriticalChance +50–100 (aditivo)
ExtraSpeed	15 s	<code>MovementSpeed</code> ×2–4, <code>JumpHeight</code> ×2–3. <code>DashCharges</code> y <code>AirJumps</code> suman 2× lo que ya se tenía (solo si es > 0)
ExtraScavenging	7.5 s	<code>PickupRange</code> ×2–4, <code>Scavenging</code> +25–50, <code>DoubleDrop</code> +25–75

Tipo	Duración	Efecto ()
Invulnerability	7.5 s	PlayerHealth.GrantInvulnerability más el flag propio IsPowerupInvulnerable, que consulta PlayerHealth.cs:287
FullHeal	instantáneo	Vida al máximo, recarga de munición manual y reset del cooldown de la habilidad activa (:154-167)
Nuke	instantáneo	50–500 de daño en un radio de 20–40 m. Daño pleno hasta r/2, caída lineal hasta 0 en r. Knockback 18 × falloff (:169-191)

Reglas:

- Solo hay **un buff temporizado activo**. ApplyTimedBuff llama primero a ClearBuff (:195), así que agarrar ExtraSpeed durante ExtraDamage cancela el daño. La invulnerabilidad corre en un timer aparte y convive con un buff.
- Los modificadores entran como StatUpgradeSource.TemporaryPowerup con una fuente única (_buffSource) y se sacan en bloque con RemoveModifiersFromSource.
- Hay FX de partículas creadas por código, con el color del power-up. Bajan la emisión en los últimos 2.5 s.
- En test_balance, LogStatDeltas loguea el antes y el después de cada stat que tocó.

Scrap dentro de la run

EnemyScrapDrop.OnEnemyDied (EnemyScrapDrop.cs:22-44):

Enemigo	Resultado
Boss (WeaponEnemyClassifier.GetKind == Boss)	+5–10 Scrap directo al save (Random.Range(5, 11)) y SaveNow(). No hay pickup
Resto	Chance $0.001 \times (\text{Scavenging} / 50)$, o sea 0.1 % con la base. Spawnea ScrapPickup

ScrapPickup.OnPickedUp suma 1 Scrap y guarda el archivo en el acto (ScrapPickup.cs:24-25). El Scrap recogido persiste aunque la run termine en derrota. El pool (ScrapPickupPool) se autocrea con 8 instancias.

BossManager no tiene lógica de drop propia. Solo engancha los componentes de arriba y su handler de muerte (OnBossInstanceDied, BossManager.cs:316-328), que avisa OnBossDefeated.

Liquidación al terminar la run

GameManager.EnterEndState → ReportRunToSaveSystem(victory) (GameManager.cs:98-104, :129-140) → SaveManager.ReportRunEnded. Corre tanto en victoria como en derrota.

CalculateScrapEarned (GameManager.cs:142-155):

```
scrap = round(segundosSobrevividos / 10) + bossKills × 25
      + Σ materiales comunes en inventario × 1
      + Σ materiales raros en inventario × 5
```

Ejemplo: 15 min, 2 bosses, 120 comunes y 10 raros sin gastar dan $90 + 50 + 120 + 50 = \mathbf{310 \text{ Scrap}}$. El comentario del código lo marca como fórmula placeholder.

Resumen de fuentes de Scrap por boss: 5–10 al morir (EnemyScrapDrop) más 25 en la liquidación.

Otras fuentes: recompensas de logros (ver doc 22) y el botón DEV, que fija 9999.

Deuda / puntos abiertos

- **ExtraDamage suma +50 a +100 a CriticalChance**, un stat que va de 0 a 1 (`TemporaryPowerupController.cs:109`). Durante 15 s el crítico queda en 100 %. Es el mismo bug de escala que el doc 05. La definición de level-up ya se corrigió (0.01), pero el power-up no.
- ExtraScavenging usa la misma escala "de a 100" (+25–50 Scavenging, +25–75 DoubleDrop). Con la fórmula real de `EnemyMaterialDrop` (ver doc 20), los materiales ya caen al 100 %. El DoubleDrop pasa de –30 a positivo y se clampea a 100 %: el buff termina siendo "doble drop garantizado" más imán. Además sube la chance de power-up y de Scrap ($\times 1.5$ –2).
- Nuke llama a `enemy.ApplyDamage` directo, sin pasar por `WeaponDamageApplier`: no hay críticos, multiplicadores de daño ni `NotifyDamageInstance`. La muerte igual dispara `OnDied`, así que el nuke genera drops (y power-ups en cadena).
- `ScrapPickup` y el Scrap de boss escriben el JSON en cada pickup. Junto con los reportes por kill del doc 22, son escrituras a disco frecuentes en el hilo principal.
- En build, los power-ups se ven como esferas: los prefabs authored solo se resuelven en editor.
- El tipo de power-up es uniforme. FullHeal y Nuke salen con la misma frecuencia que los buffs, sin control de diseño.
- El boss suma Scrap por dos caminos (5–10 inmediato + 25 al final). Hay que confirmar si eso es intencional.

8 Level-up y stats automáticas

Cómo se gana XP, cuánto pide cada nivel y qué pasa en cada level-up. En cada nivel hay una carta de pasivos a elegir y una ruleta automática de stats, que se resuelven en cola. Las armas no se ganan por level-up: salen de la estación de crafting (doc 20).

Scripts

- `Player/PlayerXP.cs`: XP acumulada, curva, tope y evento `OnLevelUp`.
- `XP/LevelUpOrchestrator.cs`: cola de niveles pendientes. Coordina los handlers.
- `Player/Passives/PassiveItemLevelUpHandler.cs` + `PassiveItemRoulette.cs` + `XP/DynamicRoulette.cs`: carta de pasivos.
- `Player/PlayerStats/PlayerStatsLevelUpHandler.cs` (incluye `StatMath`): ruleta automática de stats.
- `XP/LevelUpChoiceUI.cs`: UI de la carta (pausa con `timeScale = 0`).
- `XP/LevelUpStatFeedback.cs`: textos flotantes con las stats que subieron.
- `Weapon/Managers/WeaponLevelUpHandler.cs`: carta de armas. Está en el prefab, pero **no se invoca**.
- Legacy: `XP/UpgradeManager.cs` (vacío, marcado como reemplazado) y `XP/XPDrop.cs`, `XPXPickup.cs`, `XPXPool.cs` (orbes). `GameplayScene` tiene un `XPXPool`, pero ningún enemigo lleva `XPDrop` y ningún script llama al pool.

Todos los componentes están en `Assets/Prefabs/player.prefab`, y el orquestador tiene sus cuatro referencias serializadas.

Entrada de XP

La única fuente es recoger un material: `MaterialPickupReceiver.GrantMaterial` (`MaterialPickupReceiver.cs:39-43`) suma `GetPickupXpValue × amount`. Un común vale 1 y un raro vale 5 (`MaterialCatalog.cs:54-55`). Un doble drop duplica la XP.

`PlayerXP.AddExperience` (`PlayerXP.cs:58-84`) resta el requisito en un loop y dispara un `OnLevelUp` por cada nivel. Un solo pickup puede dar varios niveles. Al llegar al tope, la XP sobrante se descarta.

Curva de XP

Valores serializados en `player.prefab`, iguales a los defaults: primer requisito 10, multiplicador 1.2, tope 36. `GameplayScene` no los sobrescribe.

Requisito para pasar del nivel L al L+1: $\text{req}(1) = 10$, $\text{req}(L) = \text{ceil}(\text{req}(L-1) \times 1.2)$ (`PlayerXP.cs:86-96`). La cuenta es en float de 32 bits, igual que en Unity. Por redondeo float, del nivel 21 en adelante difiere en +1 de la cuenta exacta.

L→L+1	XP	Acum.	L→L+1	XP	Acum.	L→L+1	XP	Acum.
1→2	10	10	13→14	101	538	25→26	925	5 446
2→3	12	22	14→15	122	660	26→27	1 110	6 556
3→4	15	37	15→16	147	807	27→28	1 332	7 888
4→5	18	55	16→17	177	984	28→29	1 599	9 487
5→6	22	77	17→18	213	1 197	29→30	1 919	11 406
6→7	27	104	18→19	256	1 453	30→31	2 303	13 709
7→8	33	137	19→20	308	1 761	31→32	2 764	16 473
8→9	40	177	20→21	370	2 131	32→33	3 317	19 790

L→L+1	XP	Acum.	L→L+1	XP	Acum.	L→L+1	XP	Acum.
9→10	48	225	21→22	445	2 576	33→34	3 981	23 771
10→11	58	283	22→23	534	3 110	34→35	4 778	28 549
11→12	70	353	23→24	641	3 751	35→36	5 734	34 283
12→13	84	437	24→25	770	4 521			

Llegar al 36 pide 34 283 XP, que son unos 34 000 comunes o 6 900 raros. Con el drop al 100 % que da la fórmula real de Scavenging (doc 20), la XP cae en casi cada kill.

Orquestador

LevelUpOrchestrator (LevelUpOrchestrator.cs:43-74):

1. OnLevelUp(n) encola n. Si no hay proceso en curso, arranca la corrutina.
2. Por cada nivel en la cola, en orden:
3. PassiveItemLevelUpHandler.PresentAndApplyCoroutine(n): la carta pausa el juego y espera la elección.
4. PlayerStatsLevelUpHandler.ApplyLevelUpStats(n): la ruleta de stats es inmediata.
5. LevelUpStatFeedback.Show(resultados): encola un lote de textos flotantes (1.4 s por mensaje).
6. _weaponHandler está serializado y resuelto en Awake, pero no se usa.

Carta de pasivos

PassiveItemLevelUpHandler: _choicesOffered = 3 en el prefab. El pool tiene los 17 pasivos, filtrados por SaveManager.IsUnlocked (:28-38).

- Elegibles por slot (PassiveItemManager.cs:174-218): si el slot está lleno, solo las mejoras de los equipados con `Level < MaxLevel` (6 en los 17 assets; el default de script es 5). Si no está lleno, los ítems del slot que no estén equipados, más las mejoras de los equipados.
- Ruleta (PassiveItemRoulette.cs): cada clave (ítem nuevo o mejora de una instancia) arranca con peso 5. Se sacan 3 distintos por peso. El elegido baja 1 (mínimo 1) y los otros dos ofrecidos suben 1. Lo no ofrecido no cambia.
- Si no hay elegibles, se loguea un warning y el nivel sigue solo con stats.

Ruleta automática de stats

PlayerStatsLevelUpHandler (_levelCap = 36 serializado, duplicado del de PlayerXP).

Stats elegibles: las StatDefinition con UpgradeableByLevel = 1. Todas arrancan con peso 5 (:20-31).

Tiradas por nivel (GetUpgradeCountForLevel, :117-127):

Nivel alcanzado	2-5	6-10	11-15	16-20	21-25	26-29	30-35	36
Tiradas	5	6	7	8	9	10	11	20

En total son 296 tiradas del nivel 2 al 36.

Pesos (:33-54, :107-115): cada tirada sortea por peso y la stat elegida baja 1 en el acto (mínimo 1). La misma stat puede salir varias veces en un nivel. Al cerrar el nivel, las que no salieron suben 1.

Monto por tirada (:78-105, StatMath, :130-141):

```
monto = LevelUpgradeBaseAmount × (1 + nivel / 18) × U(0.9, 1.1) × crecimientoMeta
```

`1 + nivel/18` va de 1.11 (nivel 2) a 3.0 (nivel 36). `crecimientoMeta` es $\times 1$, $\times 1.15$ desde el nivel meta 5 de esa stat y $\times 1.3225$ en el 10 (`SaveManager.cs:366-375`). Entra como modificador **aditivo** `StatUpgradeSource.LevelUp`. Si la stat es `MaxHealth`, además suma `round(monto)` a la vida actual.

Stat (asset)	Base	Base stat	Por tirada nv 2 / nv 36	Esperado al 36*
MaxHealth	5	100	5.6 / 15	+273
DamageMultiplier	0.04	1	0.044 / 0.12	+2.19
AttackSpeedMultiplier	0.03	1	0.033 / 0.09	+1.64
ProjectileAreaSize	0.03	1	0.033 / 0.09	+1.64
MovementSpeed	0.04	6.8	0.044 / 0.12	+2.19
PickUpRange	0.2	5	0.22 / 0.6	+10.9
CriticalChance (0–1)	0.01	0	0.011 / 0.03	+0.55
DamageResistance (0–1)	0.006	0	0.007 / 0.018	+0.33
AbilityCooldownReduction (0–1)	0.02	0	0.022 / 0.06	+1.09
AmmoMultiplier	0.01	1	0.011 / 0.03	+0.55
DashSpeed	0.025	10	0.028 / 0.075	+1.37
JumpHeight	0.0025	3	0.003 / 0.0075	+0.14

* Estimación: las 296 tiradas se reparten parejo entre las 12 stats (los pesos tienden a igualarse), sin meta y con factor aleatorio medio 1. Suma de escalas $\approx 656 / 12 \approx 54.7 \times$ base.

El crecimiento meta solo tiene efecto en las 6 stats de la tienda que además son de level-up: `DamageMultiplier`, `AttackSpeedMultiplier`, `ProjectileAreaSize`, `CriticalChance`, `MaxHealth` y `PickupRange`.

WeaponLevelUpHandler

Implementado: ofrece 2 o 3 cartas de armas ("Elige un arma"), con pool, pesos y descripciones. Está en `player.prefab` y tiene referencia en el orquestador, pero `ProcessSingleLevelUp` no lo llama. Queda por diseño: el loadout se decide al inicio de la run y en crafting (doc 05).

Deuda / puntos abiertos

- La XP por material es 1/5, no 4/12 como en el DEV del 21-7. Con 34 283 XP para el tope, hay que medir el tiempo-a-nivel real antes de tocar la curva.
- AbilityCooldownReduction** suma $\approx +1.09$ esperado al nivel 36 en un stat de 0 a 1. El consumidor lo clampéa a 0.95 (`WeaponMath.cs:109`), así que cerca del tope de nivel la habilidad queda con el 5 % del cooldown. `DamageResistance` (clamp 0.95, `PlayerStatMath.cs:10`) llega a ≈ 0.33 solo con level-ups, antes de sumar pasivos.
- `_levelCap` está duplicado en `PlayerXP` y en `PlayerStatsLevelUpHandler`. `TemporaryPowerupController` usa el de `PlayerXP`. Si divergen, el escalado se desalinea.
- `MaxHealth` agrega el monto sin redondear al stat, pero a la vida actual le suma `round(monto)`. Puede quedar un desfase fraccional entre vida y máximo.
- El título de la carta está en español ("Elige un objeto" / "Elige un arma"), con el resto de la UI de meta en inglés.
- Hay restos legacy: `UpgradeManager`, y `XPPool` en `GameplayScene` junto con el prefab `xp_drop`, sin uso.
- El salto de 11 a 20 tiradas en el nivel 36 es el único nivel con ese volumen. Como el nivel 36 es el tope, esas 20 tiradas pasan una sola vez, al final.

9 Stats del jugador

Cada stat del jugador es un `StatDefinition` (`ScriptableObject`) con valor base. En runtime se le apilan modificadores aditivos y multiplicativos de level-up, pasivos, powerups, meta y efectos. `PlayerStats` es la única fuente del valor actual y los consumidores (movimiento, armas, vida, drops) lo leen bajo demanda. La vida actual, los escudos y los i-frames viven aparte, en `PlayerHealth`.

Scripts

Script	Rol
<code>Assets/Scripts/Player/PlayerStats/StatsCore.cs</code>	Enums (<code>StatType</code> , <code>StatCategory</code> , <code>StatUpgradeSource</code> , <code>StatModifierType</code>), <code>StatDefinition</code> , <code>StatModifier</code> , <code>RuntimeStat</code>
<code>Assets/Scripts/Player/PlayerStats/PlayerStats.cs</code>	Mapa <code>StatType</code> → <code>RuntimeStat</code> , alta y baja de modificadores, evento <code>OnStatChanged</code>
<code>Assets/Scripts/Player/PlayerStats/PlayerStatMath.cs</code>	Clamps de consumo: resistencia, regen, lifesteal, pickup, elite, drops
<code>Assets/Scripts/Player/PlayerStats/PlayerStatsLevelUpHandler.cs</code>	Ruleta automática de stats por nivel (<code>StatMath</code>)
<code>Assets/Scripts/Player/PlayerStats/StatDisplayFormat.cs</code>	Formato y detección de saturación para UI
<code>Assets/Scripts/Player/PlayerHealth.cs</code>	Vida, daño, resistencia, regen, escudo, burn, i-frames, muerte
<code>Assets/Scripts/Meta/MetaProgressionApplicator.cs</code>	Multiplicadores de meta al arrancar la run
<code>Assets/Scripts/Weapon/UI/StatAttributionPanel.cs</code>	Vista dev en la pausa: atribución por fuente y último golpe
<code>Assets/Scripts/Weapon/UI/LastHitRecorder.cs</code>	Guarda el último <code>WeaponDamageRoll</code> resuelto

StatType

`StatsCore.cs:5-17`. 30 valores, en orden: `MovementSpeed`, `JumpHeight`, `AirJumps`, `DashCharges`, `DashSpeed`, `DamageMultiplier`, `DamageFlat`, `EliteDamageMultiplier`, `AttackSpeedMultiplier`, `ProjectileAreaSize`, `CriticalChance`, `CriticalDamage`, `Knockback`, `AmmoMultiplier`, `MaxHealth`, `HealthRegeneration`, `Lifesteal`, `DamageResistance`, `PickupRange`, `ExtraEliteChance`, `Scavenging`, `DoubleDrop`, `BaseFireInterval`, `AbilityDamageMultiplier`, `AbilityCooldownReduction`, `LongRangeDamageMultiplier`, `CloseRangeDamageMultiplier`, `ShieldCharges`, `ShieldRechargeDelay`, `HealthRegenerationDelayReduction`.

`StatCategory`: Mobility, Offensive, Defensive, Miscellaneous (:19).

Fórmula

`RuntimeStat.CurrentValue` (`StatsCore.cs:123-141`):

$\text{valor} = (\text{base} + \sum \text{aditivos}) \times \prod \text{multiplicativos}$, con floor si `IsInteger`.

- La base es `BaseValue` del asset, o un override. `PlayerStats.Awake` pisa la base de `MaxHealth` con `PlayerHealth._maxHealth` (`PlayerStats.cs:25-33`).
- Un multiplicativo de 1 es neutro. Un multiplicativo afecta también a todos los aditivos, no solo a la base.
- `RuntimeStat` no aplica clamps. Los aplica cada consumidor:

Stat	Clamp	Dónde
DamageResistance	0–0,95; daño mínimo 1	PlayerStatMath.cs:5-12
Lifesteal	0–1	PlayerStatMath.cs:27
ExtraEliteChance	0–0,95	PlayerStatMath.cs:41
HealthRegeneration, PickupRange	≥ 0	PlayerStatMath.cs:14-37
Scavenging	/100, 0–1 = prob. de drop	PlayerStatMath.cs:73
DoubleDrop	(DoubleDrop + (Scavenging – base)) / 100, 0–1	PlayerStatMath.cs:76-87
CriticalChance	0–1	WeaponDamageResolver.cs:414
AbilityCooldownReduction	0–0,95	WeaponMath.cs:109
MovementSpeed	≥ 0,1	PlayerStats.cs:141
BaseFireInterval / AttackSpeedMultiplier	intervalo ≥ 0,05 / mult ≥ 0,01	PlayerStats.cs:133-138

GetStat devuelve 0 y loguea un warning si falta la definición (PlayerStats.cs:54-61). GetStatInt hace floor.

Fuentes de modificadores

StatUpgradeSource (StatsCore.cs:20):

Fuente	Quién la usa	Tipo	SourceReference
Base	MetaProgressionApplier	Multiplicativo $1 + 0,05 \times$ nivel meta	objeto privado
LevelUp	PlayerStatsLevelUpHandler; PlayerStats.ApplyUpgrade (legado)	Aditivo	null / Upgrade
PassiveItem	PassiveItemManager	Según el pasivo	instancia del pasivo
Weapon	Nadie en el código actual	—	—
TemporaryEffect	OverheatManager → SetRuntimeFireRateMultiplier	Aditivo (mult – 1) sobre AttackSpeedMultiplier	PlayerStats
TemporaryPowerup	TemporaryPowerupController	Ambos	_buffSource

Las bajas se hacen por referencia (RemoveModifiersFromSource) o por categoría (ClearModifiersFromSourceType) (PlayerStats.cs:105-120). Los dos métodos vuelven a emitir OnStatChanged para todos los stats.

Assets de stats

Assets/ScriptableObjects/PlayerSO/Stats/*.asset. Los 30 están en _statDefinitions del prefab (player.prefab:705+) y la escena no los sobrescribe. Cat: M/O/D/X = Mobility/Offensive/Defensive/Misc. Nivel = UpgradeableByLevel. Items = UpgradeableByItems. Monto = LevelUpgradeBaseAmount.

Asset	StatType	Cat	Base	Nivel	Items	Monto	%	Int
MovementSpeed	0	M	6,8	Sí	Sí	0,04		
JumpHeight	1	M	3	Sí	Sí	0,0025		

Asset	StatType	Cat	Base	Nivel	Items	Monto	%	Int
AirJumps	2	M	0		Sí	0		Sí
DashCharges	3	M	1		Sí	0		Sí
DashSpeed	4	M	10	Sí	Sí	0,025		
DamageMultiplier	5	O	1	Sí	Sí	0,04		
DamageFlat	6	M	0			0		
EliteDamageMultiplier	7	O	1,25		Sí	1,2		
AttackSpeedMultiplier	8	O	1	Sí	Sí	0,03		
ProjectileAreaSize	9	O	1	Sí	Sí	0,03		
CriticalChance	10	O	0	Sí	Sí	0,01	Sí	
CriticalDamage	11	O	2,5		Sí	0		
Knockback	12	O	1		Sí	0		
AmmoMultiplier	13	O	1	Sí	Sí	0,01		
MaxHealth	14	D	100	Sí	Sí	5		
HealthRegeneration	15	D	0,5		Sí	1,2		
Lifesteal	16	D	0		Sí	0		
DamageResistance	17	D	0	Sí		0,006	Sí	
PickUpRange	18	X	5	Sí		0,2		
ExtraEliteChance	19	X	0		Sí	0		
Scavenging	20	X	50		Sí	0		
DoubleDrop	21	X	-30		Sí	0		
BaseFireInterval	22	M	0			0		
AbilityDamageMultiplier	23	O	1		Sí	0		
AbilityCooldownReduction	24	O	0	Sí	Sí	0,02	Sí	
LongRangeDamageMultiplier	25	O	1		Sí	0		
CloseRangeDamageMultiplier	26	O	1		Sí	0		
ShieldCharges	27	D	0		Sí	0		Sí
ShieldRechargeDelay	28	D	0		Sí	0		
HealthRegenerationDelayReduction	29	D	0		Sí	0		

Entran en la ruleta 12 stats. `CriticalChance` estaba en 1 por nivel y pasó a 0,01 en Balance 0.3.2 (dd551fe): el bug de datos del level-up está corregido. `DoubleDrop` -30 junto con `Scavenging` 50 significa que no hay doble drop hasta tener Scavenging por encima de 80.

Level-up automático

`PlayerStatsLevelUpHandler`, tope 36 (`player.prefab:981`).

1. Al iniciar, cada stat con `UpgradeableByLevel` recibe peso 5 (:29).

- Por nivel se hacen N tiradas: 5 (niveles 1–5), 6 (6–10), 7 (11–15), 8 (16–20), 9 (21–25), 10 (26–29), 11 (30–35), 20 (36) (:117-127). Un mismo stat puede salir más de una vez.
- Monto por tirada = $\text{LevelUpgradeBaseAmount} \times (1 + \text{nivel}/18) \times U(0,9; 1,1) \times \text{crecimiento meta}$ (:84-89, :132-139). El crecimiento meta es $\times 1,15$ desde el nivel meta 5 y $\times 1,15^2$ en el 10 (SaveManager.cs:366-375).
- Se agrega como aditivo LevelUp con label "LvN roll (base X)" (:99). Si el stat es MaxHealth, además suma $\text{round}(\text{monto})$ a la vida máxima de PlayerHealth (:101-102).
- La elegida baja 1 de peso por tirada (mínimo 1). Las que no salieron en el nivel suben 1 (:49, :107-115).

PlayerHealth

Valores: `_maxHealth` 100 (prefab); `_hitInvulnerabilitySeconds` default 1,5, prefab 1,5, **escena 0,5**; `_regenerationDelaySeconds` 5 (player.prefab:654-658).

Daño directo, `TakeDamage` (PlayerHealth.cs:282-325):

- Ignora si está muerto, con invulnerabilidad de powerup o dentro de los i-frames.
- Si hay carga de escudo, consume una, da i-frames y no quita vida.
- Aplica resistencia ($\text{round}(\text{daño} \times (1 - \text{res}))$), mínimo 1), resta, da i-frames, suena el hurt, emite `OnHitDamageTaken` y `OnHealthChanged`.
- Con 0 HP: `_isDead`, `ChallengeProgressTracker.NotifyPlayerDied`, `OnPlayerDied` (lo escucha GameManager).

Resto:

Mecánica	Comportamiento	Línea
i-frames	Mientras duran, <code>Physics.IgnoreCollision</code> contra todos los colliders de enemigos. Se re-sincroniza cada <code>Update</code>	:359-395
Burn	DoT en ticks de 0,5 s. Ignora i-frames y escudo; aplica resistencia. Refresca duración y conserva el dps mayor	:196-251
Regen	<code>HealthRegeneration</code> HP/s tras 5 - <code>HealthRegenerationDelayReduction</code> s sin daño. Acumula fracciones	:327-351
Escudo	<code>SetShieldConfig</code> desde <code>PassiveItemManager</code> (<code>ShieldCharges</code> , <code>ShieldRechargeDelay</code>). Recarga todas las cargas juntas tras el delay sin daño	:118-129, 184-194
Vida máx.	<code>ApplyMaxHealthDelta</code> : si sube, cura lo ganado; si baja, recorta	:90-105
<code>OnEnable</code>	Resetea vida al máximo, burn e i-frames	:150-160

Meta

`MetaProgressionApplier` lo agrega en runtime `SpecMetaBootstrap.cs:93-94`, porque no está en el prefab. En `Start`, para `DamageMultiplier`, `AttackSpeedMultiplier`, `ProjectileAreaSize`, `CriticalChance`, `CriticalDamage`, `MaxHealth`, `HealthRegeneration` y `PickupRange` agrega un multiplicativo $1 + 0,05 \times \text{nivel}$ (0–10, tope $\times 1,5$). Después ajusta la vida máxima por la diferencia (`MetaProgressionApplier.cs:10-20, 29-66`). La tienda de meta lo re-aplica (`MetaUpgradeShopUI.cs:613`).

Herramientas dev

Solo compilan en `UNITY_EDITOR` || `DEVELOPMENT_BUILD`.

- `StatAttributionPanel`: `PauseMenuUI.cs:80-91` lo monta en `PlayerStatsPanel` al abrir la pausa. Agrega pestañas **STATS** y **LAST HIT** junto al resumen. STATS lista todos los stats por categoría con base, subtotal de level-ups y de pasivos (con cantidad), otros, multiplicador y final. Al elegir uno muestra modificador por modificador con su label. Los stats saturados se pintan en naranja con `[!] consumers clamp this to ...`

(`StatAttributionPanel.cs:611-625,702-710`). LAST HIT desarma el último golpe en factores (base, nivel de arma, path, stat de daño, habilidad, crítico, elite, rango, escalas) y enlaza al stat correspondiente. Escape cierra primero la pestaña dev.

- `LastHitRecorder`: estático. Se suscribe una vez a `WeaponDamageResolver.OnDamageResolved` en `SubsystemRegistration` y guarda el último roll, para que se pueda ver después de pausar (`LastHitRecorder.cs:18-35`).

BalanceAutoImporter

`Assets/Scripts/Economy/Editor/BalanceAutoImporter.cs` no toca los stats del jugador. Es `[InitializeOnLoad]`, pero solo corre `ImportAll` y `CreateDefaultDropConfigs` si falta `MaterialUsageBalance.asset` (:17-30). Esos pasos escriben assets de economía y armas desde `balance_material_usage.csv` y `balance_weapon_stats.csv`. Ningún importer ni CSV escribe `PlayerSO/Stats`.

Deuda / puntos abiertos

- El powerup `ExtraDamage` suma 50–100 **aditivos** a `CriticalChance` (`TemporaryPowerupController.cs:109`), y el resolver clampéa a 0–1 (`WeaponDamageResolver.cs:414`). Durante el buff el crítico queda en 100 % a cualquier nivel. El valor debería ser 0,5–1,0 o, mejor, un aditivo chico.
- El meta de `CriticalChance` es multiplicativo sobre una base 0 (`MetaProgressionApplier.cs:51-57`): solo escala lo que dieron los level-ups (por ejemplo 0,05 → 0,075 en meta 10). Es un upgrade de tienda casi sin efecto.
- `PlayerHealth._maxHealth` se desincroniza de `GetMaxHealthTotal()`. El level-up suma `round(monto)` a la vida (`PlayerStatsLevelUpHandler.cs:101-102`), pero el stat sube `monto × multiplicador meta` y con fracciones. `PassiveItemManager` sincroniza por delta del stat, así que no corrige la deriva acumulada.
- Los modificadores de level-up tienen `SourceReference = null` (`PlayerStatsLevelUpHandler.cs:99`). `RemoveModifiersFromSource(null)` (`StatsCore.cs:150`) borraría todos los level-ups y no hay guarda en `PlayerStats.cs:105`.
- Datos muertos: `EliteDamageMultiplier` y `HealthRegeneration` tienen monto de nivel 1,2 con `UpgradeableByLevel = 0`. `UpgradeableByItems` no se lee en ningún script. `StatUpgradeSource.Weapon` no se usa.
- Montos de nivel casi nulos: `JumpHeight` 0,0025 (base 3) y `DashSpeed` 0,025 (base 10). Se llevan tiradas de la ruleta sin efecto perceptible.
- `StatDisplayFormat.GetEffectiveMaximum` (:74-84) no incluye `AbilityCooldownReduction` (clamp 0,95 en `WeaponMath.cs:109`). El panel dev nunca la marca como saturada.
- `ShieldRechargeDelay` base 0: con escudo equipado, sin delay de pasivo, las cargas vuelven todas en el frame siguiente al golpe (`PlayerHealth.cs:184-194`). Además, la escena baja los i-frames a 0,5 s contra 1,5 s en código y prefab: hay que decidir cuál es el valor de diseño.

10 Meta, challenges y logros

La progresión fuera de la run: el save en JSON, los challenges y logros que desbloquean pasivos y Path B, la tienda de upgrades meta pagada con Scrap y el menú de objetivos. Todo pasa por `SaveManager`. Los demás scripts reportan eventos o leen multiplicadores.

Scripts

- `Meta/SaveManager.cs` + `Meta/SaveData.cs`: persistencia, evaluación de logros y compras.
- `Meta/SpecMetaBootstrap.cs`: registra los challenges de la Spec y las cartas de Path B, y agrega componentes al jugador.
- `Meta/ChallengeProgressTracker.cs`: escucha daño, muertes y kills, y reporta progreso.
- `Meta/MetaProgressionApplier.cs`: aplica los upgrades meta de stats al empezar la run.
- `Meta/MetaStatUpgradeCosts.cs` (`Resources/Meta/MetaStatUpgradeCosts.asset`): precios.
- `Meta/AchievementDefinition.cs`, `AchievementConditionType.cs`, `UnlockCatalog.cs`, `UnlockRequirement.cs`, `WeaponPathUnlockData.cs`, `IUnlockable.cs`.
- `Meta/UI/ObjectivesMenuUI.cs`, `MetaUpgradeShopUI.cs`, `ObjectivesFilterChipBar.cs`, `AchievementUnlockToast.cs`, `AchievementUnlockToastView.cs`.

SaveManager

Se autocrea antes de cargar la escena (`SaveManager.cs:34-43`), con orden `-150` y `DontDestroyOnLoad`.

Archivo: `Application.persistentDataPath/scrapwaves_save.json` (:15, :55). En Windows:

`%USERPROFILE%\AppData\LocalLow\Initial Dreams\Dumpster Fire\`. Se serializa con `JsonUtility` en modo `pretty print` (:547). Si falla la lectura, arranca un save vacío y loguea un warning (:525-541).

`SaveData` (versión 3, `SaveData.cs:41-78`):

Campo	Tipo	Uso
<code>Version</code>	<code>int</code>	3. Sanitize solo la sube, sin migración
<code>Scrap</code>	<code>int</code>	Moneda meta
<code>UnlockedIds</code>	<code>string[]</code>	Pasivos, armas y <code>WeaponPath_<id>_PathB</code> desbloqueados
<code>UnlockedAchievementIds</code>	<code>string[]</code>	Logros completados
<code>PresentationAccessibility</code>	objeto	Motion, shake, flash y combat text. Sobrevive al reset
<code>TotalBossKills</code> , <code>TotalRunsCompleted</code> , <code>TotalEnemiesKilled</code>	<code>int</code>	Contadores cross-run (se cargan al terminar la run)
<code>BestSurvivalTimeSeconds</code>	<code>float</code>	Mejor tiempo de una run
<code>HighestPlayerLevel</code>	<code>int</code>	Se actualiza en cada level-up que supera el récord
<code>TotalEliteOrBossKills</code> , <code>TotalDropsLooted</code>	<code>int</code>	Contadores de challenges
<code>WeaponLevels</code>	<code>{WeaponId, HighestLevel}[]</code>	Sin call site (<code>ReportWeaponLevelReached</code> no se llama)
<code>CustomProgress</code>	<code>{Key, Value}[]</code>	Custom y RunChallenge: guarda el máximo
<code>WeaponKills</code>	<code>{WeaponId, Kills}[]</code>	Kills por arma (crédito al último golpe)

Campo	Tipo	Uso
MetaStatLevels	{StatTypeName, Level}[]	Nivel meta 0–10 por stat
MetaItemUpgrades	{UnlockId, Level}[]	Nivel meta 0–3 por pasivo

Evaluación (`SaveManager.cs:499-523`): recorre el catálogo y marca como completo todo logro con `GetProgress + 0.0001 ≥ TargetValue`. Suma `ScrapReward`, desbloquea cada `RewardUnlockId` y dispara `OnAchievementUnlocked`. La llaman todos los `Report*`.

Catálogo: `Resources.LoadAll("Meta/Achievements")` (:69) más los logros runtime de `SpecMetaBootstrap`, deduplicados por id (:269-295).

Challenges registrados

Todos se crean en `SpecMetaBootstrap.EnsureRegistered` (`SpecMetaBootstrap.cs:23-77`).

Id	Condición	Objetivo	Recompensa
first_kill	Custom first_kill (cualquier kill)	1	10 Scrap
first_death	Custom first_death	1	10 Scrap
first_extraction	RunsCompletedTotal (victorias)	1	10 Scrap
thaw_them_out	WeaponKillsTotal, Flamethrower	2500	50 Scrap + Path B Flamethrower
to_little_pieces	WeaponKillsTotal, RocketLauncher	2500	50 + Path B RocketLauncher
make_it_rain	WeaponKillsTotal, Mortar	2500	50 + Path B Mortar
go_for_the_head	WeaponKillsTotal, AutomaticCannon	2500	50 + Path B AutomaticCannon
studied_the_blade	WeaponKillsTotal, RotatingBlade	2500	50 + Path B RotatingBlade
high_value_targets	EliteOrBossKillsTotal	250	40 + Shop_Head_BountyHunterModule
resourcefulness	DropsLootedTotal	10000	40 + Shop_Head_ScavengerModule
not_how_you_heal	RunChallenge: de 100 % a 0 % de vida en ≤ 5 s	1	30 + Shop_Core_ScrapReassembly
survival_adept	RunChallenge: 120 s sin recibir daño	120	30 + Shop_Core_ElectromagneticShield
one_shot_one_kill	RunChallenge: un golpe de ≥ 1000 de daño	1000	30 + Shop_Arm_AdvancedTargetingModule

Assets en `Resources/Meta/Achievements`:

Asset	Condición	Objetivo	Recompensa
BossHunter	BossKillsTotal	3	50 Scrap, _rewardUnlockIds: []
RustyMarathoner	SurviveTimeSingleRun	600 s	50 Scrap, _rewardUnlockIds: []

Los 5 pasivos con `_unlockedFromStart: 0` son justo los 5 que se ganan por challenge. Los otros 12 arrancan desbloqueados. Todos tienen `ScrapPrice: 0` y ningún `RequiredAchievement`.

Path B: `RegisterPathUnlockCards` (`SpecMetaBootstrap.cs:97-114`) crea un `WeaponPathUnlockData` runtime por arma, con **50 Scrap** de precio y sin logro requerido. Se compra en la tienda o se gana con el challenge del arma. `SaveManager.IsPathUnlocked` siempre deja pasar PathA (`SaveManager.cs:444-451`).

ChallengeProgressTracker

Singleton autocreado `AfterSceneLoad` (`ChallengeProgressTracker.cs:149-150`).

Evento	Llamador	Qué reporta
NotifyEnemyKilled	EnemyHealth.cs:168	Kill al LastDamagingWeaponId, elite/boss, first_kill (:106-121)
NotifyDamageInstance	WeaponDamageApplier.cs:38	Máximo de la run. Reporta si ≥ 1000 (:91-104)
NotifyPlayerDamaged	PlayerHealth.cs:299/313	Resetea el contador sin daño. Chequea $100 \rightarrow 0$ en ≤ 5 s (:64-79)
NotifyPlayerDied	PlayerHealth.cs:248/322	first_death
NotifyDropLooted	MaterialPickupReceiver.cs:45	TotalDropsLooted += amount
Update	—	Si $\text{now} - \text{últimoDaño} \geq 120 \rightarrow \text{survival_adept}$ (:36-62)

MetaProgressionApplier

Lo agrega SpecMetaBootstrap al jugador y aplica en Start (MetaProgressionApplier.cs:24-67).

- Stats: DamageMultiplier, AttackSpeedMultiplier, ProjectileAreaSize, CriticalChance, CriticalDamage, MaxHealth, HealthRegeneration, PickupRange (:10-20).
- Cada nivel es un **multiplicador** $1 + 0.05 \times \text{nivel}$ (hasta $\times 1.5$ en nivel 10), con fuente StatUpgradeSource.Base (SaveManager.cs:359-363).
- Crecimiento en level-up: GetMetaStatGrowthMultiplier da $\times 1.15$ desde el nivel meta 5 y $\times 1.3225$ en el 10 (SaveManager.cs:366-375). Lo consume PlayerStatsLevelUpHandler (doc 23).
- Pasivos: GetMetaItemPowerMultiplier = 1.1^{nivel} (0-3, hasta $\times 1.331$) sobre cada bonus del pasivo (PassiveItemManager.cs:236-237). Quedan excluidos los pasivos con ShieldCharges, AirJumps o DashCharges (MetaUpgradeShopUI.cs:742-754).
- Después de aplicar, ajusta la vida actual con el delta de vida máxima.

Tienda de upgrades meta

MetaUpgradeShopUI (pestaña Upgrades). Muestra las 8 stats de arriba y los pasivos desbloqueados que se pueden mejorar.

MetaStatUpgradeCosts.asset: base 25, crecimiento 1.35; ítem base 40, crecimiento 1.5 (iguales a los defaults). La fórmula es $\text{round}(\text{base} \times \text{crecimiento}^{(n-1)})$ (MetaStatUpgradeCosts.cs:14-24).

Nivel	1	2	3	4	5	6	7	8	9	10	Total
Stat	25	34	46	62	83	112	151	204	276	372	1365
Pasivo	40	60	90								190

Llevar las 8 stats al máximo cuesta 10 920 Scrap. Los topes (10 y 3) se aplican en SaveData (SaveData.cs:86, :116).

UI

- ObjectivesMenuUI (menú principal): pestañas Objectives, Unlocks y Upgrades. Se navega con A/D o con los hombros del gamepad (ObjectivesMenuUI.cs:188-216) y recuerda la última pestaña en la sesión. En Unlocks se compra con SaveManager.TryPurchase (:956).
- Botones DEV, solo en UNITY_EDITOR || DEVELOPMENT_BUILD (:31-38, :447-510):
- "DEV: Reset progress": SaveManager.ResetProgress. Borra todo menos accesibilidad.
- "DEV: Max upgrades": MetaUpgradeShopUI.DevMaxOutAllUpgrades (stats a 10 y pasivos desbloqueados a 3) más Scrap a 9999, con una sola escritura (DevCommit).
- AchievementUnlockToast: singleton DontDestroyOnLoad autocreado (AchievementUnlockToast.cs:362). Encola los desbloques y muestra cada uno 3 s, con fade de 0.25 s en tiempo unscaled (:20-21). En TitleScreen

construye su propio panel. En gameplay usa el `AchievementUnlockToastView` de la escena y espera si no puede presentar.

Deuda / puntos abiertos

- **Escritura a disco por kill.** `ReportWeaponKill` y `ReportEliteOrBossKill` llaman a `Save()` en cada kill (`SaveManager.cs:297-317`), igual que `ReportDropsLooted` en cada pickup. Con cientos de kills por minuto, es IO sincrónico en el hilo principal.
- `ResetRunScratch` no tiene llamadores. `_lastPlayerDamageTime` y `_runMaxDamageInstance` pasan de una run a la siguiente, así que `survival_adept` puede completarse al arrancar una run si pasaron 120 s de `Time.time` desde el último daño (tiempo de menú incluido).
- Las descripciones de `BossHunter` y `RustyMarathoner` prometen "Overheated Core" y "Jet Boots". Ninguno de los dos pasivos existe y `_rewardUnlockIds` está vacío.
- Path B se puede comprar por 50 Scrap sin hacer el challenge de 2500 kills. Hay que decidir si el challenge es un atajo o un requisito (`RequiredAchievement` es null).
- `CriticalChance` recibe el meta como multiplicador sobre una base de 0, así que solo escala lo que aportan los otros aditivos. `CriticalDamage` y `HealthRegeneration` no son `UpgradeableByLevel`, y el crecimiento meta de los niveles 5 y 10 no tiene efecto en ellos.
- `WeaponLevelReached` y `ReportWeaponLevelReached` no tienen call site. `SaveData.Version` no migra datos.
- `first_extraction` cuenta victorias (`TotalRunsCompleted` solo sube con `victory`). Hay que confirmar que "extraction" y victoria son lo mismo.